

Universidad del Valle

Escuela de Ingeniería de Sistemas y Computación

Programación por Restricciones

Taller 1: Modelamiento e Implementación de CSPs

Integrantes:

Juan Diego Cárdenas Mejía - 2416437
Junior Orlando Cantor Arevalo - 2224949
Cristian David Cabrera Pantoja - 2226285

Profesor: Robinson Duque

Cali, Colombia
5 de noviembre de 2026

Índice

1. Problema 1: Sudoku **1**
- 1.1. Descripción del Problema y Abstracción CSP . . . 1
  - 1.1.1. Parámetros y Datos de Entrada . . . 1
  - 1.1.2. Variables de Decisión y Dominios . . . 2
  - 1.1.3. Restricciones del Modelo . . . 2
- 1.2. Detalles de Implementación (`sudoku.mzn`) . . . 4
  - 1.2.1. Diferentes Implementaciones de Restricciones . . . 4
  - 1.2.2. Restricciones Redundantes y Rompimiento de Simetrías . . . 5
- 1.3. Estrategias de Búsqueda y Distribución . . . 6
- 1.4. Pruebas y Análisis Experimental . . . 7
- 1.5. Conclusiones del Ejercicio . . . 9

2. Problema 2: Kakuro **11**
- 2.1. Descripción del Problema y Abstracción CSP . . . 11
  - 2.1.1. Parámetros y Datos de Entrada . . . 11
  - 2.1.2. Variables de Decisión y Dominios . . . 12
  - 2.1.3. Restricciones del Modelo . . . 12
- 2.2. Detalles de Implementación (`kakuro.mzn`) . . . 13
  - 2.2.1. Variantes en el Modelado de Restricciones . . . 13
  - 2.2.2. Restricciones Redundantes y Rompimiento de Simetrías . . . 13
- 2.3. Estrategias de Búsqueda y Distribución . . . 14
- 2.4. Pruebas y Análisis Experimental . . . 14
- 2.5. Conclusiones del Ejercicio . . . 15

3. Problema 3: Secuencia Mágica **16**
- 3.1. Descripción del Problema y Abstracción CSP . . . 16
  - 3.1.1. Parámetros y Datos de Entrada . . . 16
  - 3.1.2. Variables de Decisión y Dominios . . . 17
  - 3.1.3. Restricciones Básicas . . . 17
- 3.2. Detalles de Implementación (`secuencia.mzn`) . . . 17
  - 3.2.1. Restricciones Redundantes y Demostración Teórica . . . 18
  - 3.2.2. Rompimiento de Simetrías . . . 19
- 3.3. Estrategias de Búsqueda y Distribución . . . 20
- 3.4. Pruebas y Análisis Experimental . . . 21
- 3.5. Conclusiones del Ejercicio . . . 25

4. Problema 4: Acertijo Lógico **26**
- 4.1. Descripción del Problema y Abstracción CSP . . . 26
  - 4.1.1. Pistas del Problema . . . 26
  - 4.1.2. Parámetros y Datos de Entrada . . . 27
  - 4.1.3. Variables de Decisión y Dominios . . . 27
  - 4.1.4. Restricciones del Modelo . . . 27
- 4.2. Detalles de Implementación (`acertijo.mzn`) . . . 28
  - 4.2.1. Restricciones Redundantes . . . 28
  - 4.2.2. Rompimiento de Simetrías . . . 29
- 4.3. Estrategias de Búsqueda y Distribución . . . 29
- 4.4. Pruebas y Resultados . . . 30
- 4.5. Conclusiones del Ejercicio . . . 31

5. Problema 5: Ubicación de Personas en una Reunión **33**
5.1. Descripción del Problema y Abstracción CSP . . . 33
5.1.1. Parámetros y Estructura de Entrada . . . 33
5.1.2. Variables de Decisión y Dominios . . . 35
5.1.3. Restricciones del Modelo . . . 35
5.2. Detalles de Implementación (`reunion.mzn`) . . . 36
5.2.1. Diferentes Implementaciones de Restricciones . . . 37
5.2.2. Restricciones Redundantes y Rompimiento de Simetrías . . . 38
5.3. Estrategias de Búsqueda y Distribución . . . 39
5.4. Pruebas y Análisis Experimental . . . 40
5.5. Conclusiones del Ejercicio . . . 41

6. Problema 6: Construcción de un Rectángulo **43**
6.1. Descripción del Problema y Abstracción CSP . . . 43
6.1.1. Parámetros y Datos de Entrada . . . 43
6.1.2. Variables de Decisión y Dominios . . . 43
6.1.3. Restricciones del Modelo . . . 44
6.2. Detalles de Implementación (`rectangulo.mzn`) . . . 44
6.2.1. Restricciones Redundantes y Rompimiento de Simetrías . . . 44
6.3. Estrategias de Búsqueda y Distribución . . . 46
6.4. Pruebas y Análisis Experimental . . . 46
6.5. Conclusiones del Ejercicio . . . 47

7. Conclusiones Generales **49**
7.1. Expresividad del Modelado Declarativo y Formulación Incondicional . . . 49
7.2. El Poder Operativo de las Restricciones Redundantes . . . 49
7.3. Rompimiento de Simetrías: Supresión de la Explosión Factorial . . . 50
7.4. Interacción No Lineal entre Modelado y Estrategias de Distribución . . . 50
7.5. Síntesis Metodológica . . . 51

1. Problema 1: Sudoku

1.1. Descripción del Problema y Abstracción CSP

El sudoku es un rompecabezas matemático de colocación de números, con orígenes en el juego suizo de Çuadros Latinos"pero creado en su versión moderno por Howard Garns, un inventor de rompecabezas independiente de Connersville, Indiana. El juego apareció por primera vez en Japón en 1984, donde recibió el nombre de "Sudoku", que es la abreviatura de una expresión más larga en japonés: "Sūji wa dokushin ni kagiru", que significa "los dígitos están limitados a una ocurrencia". [**?**]

Una instancia del rompecabezas consiste de una grilla de dimensión 9×9 subdividida a su vez en 9 recuadros de 3×3. Inicialmente algunas casillas se encuentran ocupadas ya con dígitos del 1 al 9. El objetivo del rompecabezas es terminar de llenar el resto de las casillas con dígitos igualmente del 1 a 9, de forma que ningún dígito se repita por fila, columna o subrecuadro.

			9		2			
	4						5	
		2				3		
2								7
			4	5	6			
6								9
		7				8		
	3						4	
			2		7			

Figura 1: Ejemplo de instancia de rompecabezas Sudoku.

Por ello el problema del Sudoku se modeló por medio de un arreglo de dos dimensiones de variables enteras, con restricciones de igualdad sobre un arreglo de entrada que representa el estado inicial del problema. Adicionalmente se imponen las restricciones de no repetición sobre las filas y columnas del arreglo. Modelando el comportamiento de las reglas del rompecabezas.

1.1.1. Parámetros y Datos de Entrada

A fin de que el modelo sea aplicable a la totalidad de la familia de problemas de *Sudoku*, se implementó el parámetro `puzzle`. El parámetro recibe un arreglo dos dimensional de dimensión 9×9 con valores de 0.,9 al interior del mismo:

$$A = (a_{i,j})_{1 \leq i,j \leq 9} \ \ con \ \ a_{i,j} \in \{0, 1 \cdots 9\}$$

Codificada en Minizinc como:

```
array[1..9, 1..9] of 0..9: puzzle;
```

De esta manera se obtienen las 81 casillas de una instancia original del puzzle, dónde los valores al interior de la celda distintos a 0 representan un valor inicial de una instancia de un Sudoku. De esta forma se obliga al modelo por medio de restricción a mantener fijos estos valores.
Por consiguiente, las celdas iguales a 0 representan casillas vacías con posibilidad de ser llenadas con valores entre 1.,9 como parte de la búsqueda de la solución.

1.1.2. Variables de Decisión y Dominios

Como variable de decisión se empleó la variable `solution` un arreglo de dos dimensiones de tamaño 9×9 conteniendo variables de dominio entero entre el 1 y el 9:

$$S = (s_{i,j})_{1 \leq i,j \leq 9} \ con \ \ s_{i,j} \in \{1, 2 \cdots 9\}$$

Definida en Minizinc por medio de:

```
array[1..9, 1..9] of var 1..9: solution;
```

De esta forma se modela la cuadrilla que debe llenarse de números del 1 al 9, sin necesidad de tener que crear manualmente 81 variables distintas para cada casilla.

1.1.3. Restricciones del Modelo

Para modelar las reglas del rompecabezas de Sudoku se emplearon un total de cuatro restricciones:

1. **Mantener estado inicial del juego:** Durante la resolución de una instancia del Sudoku el jugador solamente rellena las casillas que no se encuentren ya inicialmente definidas. En ningún momento se permite la modificación del estado inicial del Rompecabezas.

   Por ello, en una solución válida para todas aquellas casillas no vacías en el arreglo de entrada, en el caso particular de esta implementación aquellas distintas de 0, su valor debe mantenerse en el arreglo solución. Es decir, para todo $a_{ij} \neq 0$ se debe cumplir que $a_{ij} = s_{ij}$:

   $$\forall i, j \in \{1, \cdots, 9\} : \ \ a_{i,j} \neq 0 \rightarrow s_{i,j} = a_{i,j}$$

   Esto se expresó al interior de `sudoku.mzn` por medio de un condicional y la función de agregación `forall` para obtener la conjunción de restricciones:

   ```
   constraint forall (i in 1..9, j in 1..9) (
       if puzzle[i, j] != 0 then
           solution[i, j ] = puzzle[i, j]
       else
           true
       endif
       );
   ```

   De esta forma se obliga a que al interior del arreglo de solución no se modifiquen los valores impuestos por el arreglo de entrada. Modelando la imposibilidad de cambiar el estado inicial del rompecabezas real.

2. **No repetición por filas:** Para imponerla regla de que los valores de las casillas no puedan repetirse por fila se impone la siguiente condición:

   $$\forall i \in \{1, \cdots, 9\} \ \ \forall j, k \in \{1, \cdots, 9\} : \ \ s_{i,j} \neq s_{i,k}$$

   Es decir, para cada una de las filas del arreglo de solución, cada par de elementos que pertenezcan a la misma fila, deben ser distintos. En Minizinc esta restricción se implementó por medio de nuevo de la función de agregación `forall`, de la restricción global `alldifferent` y la notación de arreglos:

```
constraint forall (i in 1..9) (
  all_different([solution[i, j] | j in 1..9])
);
```

De esta forma, se generan arreglos `solution[i, j]` con $j \in \{1, \cdots, 9\}$ lo que corresponde a una fila del arreglo de solución, y se impone la restricción que todos sus elementos deben ser distintos, por medio de la restricción `all_different`. Finalmente, se impone la conjunción de todas esas restricción para todos los $i \in \{1, \cdots, 9\}$. Efectivamente imponiendo que todas las filas deban individualmente contener elementos totalmente distintos entre sí.

3. **No repetición por columnas:** De forma equivalente, para imponer la no repetición por columnas se emplea la condición:

$$\forall j \in \{1, \cdots, 9\} \ \forall i, k \in \{1, \cdots, 9\} : \ s_{i,j} \neq s_{k,j}$$

Lo cual se codifica por medio de una restricción en también equivalente en Minizinc:

```
    constraint forall (j in 1..9) (
      all_different([solution[i, j] | i in 1..9])
);
```

Imponiendo que todas las columnas contengan únicamente elementos distintos entre sí.

4. **No repetición por bloques:** Finalmente, es necesario imponer también la no repetición de dígitos por cada sub-bloque, para ello definimos los 9 sub-bloques indexados sobre las filas y columnas $R, C \in \{0, 1, 2\}$:

$$B_{R,C} = \{s_{i,j} | 3R + 1 \leq i \leq 3R + 3, 3C + 1 \leq j \leq 3C + 3\}$$

Observe que R y C definen desplazamientos sobre las filas y columnas respectivamente para iterar sobre los nueve sub-bloques, en dónde al interior de cada uno $i, j \in \{1, 2, 3\}$.

Para cada bloque definimos entonces que todas las casillas al interior de un mismo bloque sean distintas:

$$\forall R, C \in \{0, 1, 2\} \ \forall s_{i,j}, s_{i',j'} \in B_{R,C} : i \neq i' \vee j \neq j' \rightarrow s_{i,j} \neq s_{i',j'}$$

Es decir que cada para de casillas al interior de un mismo sub-bloque deben tener valores distintos, lo que se debe cumplir para todos los 9 sub-bloques.

Aprovechando la naturaleza iterativa de la definición de los bloques, la restricción en Minizinc se codificó por medio de dos variables auxiliares de iteración `k` y `l`:

```
constraint forall (k in 0..2, l in 0..2) (
  all_different([solution [i + 3*k, j+ 3*l] | i in 1..3, j in 1..3])
);
```

De esta forma, se mantiene la misma estructura lógica: Se itera por medio de desplazamientos en las filas y columnas a través de los 9 sub-bloques, y para el arreglo de cada uno de ellos se impone la conjunción de restricciones `all_different`.

1.2. Detalles de Implementación (`sudoku.mzn`)

Al momento de implementar el modelo en Minizinc de forma adicional se tomaron las siguientes consideraciones adicionales:

1.2.1. Diferentes Implementaciones de Restricciones

Alternativamente, para tener mayor control sobre la implementación del modelo en Minizinc, las restricciones de no repetición sobre filas y columnas se puede implementar de forma manual, con dos iteradores por columna o fila respectivamente:

```
% Implementación alternativa de diferencia con misma fila
constraint forall(i in 1..9, j1, j2 in 1..9 where j1 < j2) (
    solution[i, j1] != solution[i, j2]
```

Y para la misma columna:

```
% Implementación alternativa de diferencia con misma columna
constraint forall(j in 1..9, i1, i2 in 1..9 where i1 < i2) (
    solution[i1, j] != solution[i2, j]
```

Observe para estas implementaciones es necesario establecer las condiciones adicionales $i1 < i2$ y $j1 < j2$ de forma que nunca se comparen las mismas celdas y se imponga una restricción de igualdad de una celda consigo misma:

```
    solution[i,j] != solution[i,j]
```

Pues esto resultaría siempre en un problema irresoluble.

Por otro lado, la implementación manual de la restricción por sub-bloques no resulta tan trivial. Para ello, se deben emplear cuatro variables para iterar sobre cada casilla del sub-bloque, incluyendo dos condiciones para evitar la no igualdad de la misma celda. Para ello se puede emplear la siguiente implementación:

```
constraint forall (k, l in 0..2, i1,i2,j1,j2 in 1..3 where i1 < i2 \/ j1 < j2) (
    solution[i1 + 3*k,j1 + 3*l] != solution[i2 + 3*k,j2 + 3*l]
);
```

Esta versión de la restricción cubre la totalidad de las $\binom{9}{2} = 36$ comparaciones necesarias. Aunque resulta en pares de restricciones redundantes del tipo:

```
    solution[i1,j1] != solution[i2,j2] /\
    solution[i2,j2] != solution[i1,j1]
```

Para eliminar las restricciones redundantes es posible emplear la fórmula para mapear todas las casillas del sub-bloque a un índice linear $k = 3(i-1) + j$ y así reescribir la restricción como:

```
constraint forall (k, l in 0..2, i1,i2,j1,j2 in 1..3 where (3*(i1 - 1) + j1) < (3*(i2 - 1) +
    solution[i1 + 3*k,j1 + 3*l] != solution[i2 + 3*k,j2 + 3*l]
);
```

Lo que elimina totalmente los pares de restricciones redundantes.

1.2.2. Restricciones Redundantes y Rompimiento de Simetrías

Restricciones Redundantes: Una de las técnicas más comunes para resolver el Sudoku se puede implementar como una restricción redundante para acelerar el proceso de poda de los árboles de reducción: *Regla de 45:* Dado que tanto, filas, columnas y sub-bloques deben contener todos los dígitos del 1 al 9, un conjunto común de restricciones redundantes consistente en imponer que todos estos subconjuntos sumen 45, dado que $\sum_{i=1}^{9} i = 45$. Implementados en Minizinc por medio de la expresión `sum` se expresan como:

```
% Sumas redundantes por Fila y Columna
constraint forall(i in 1..9) (
    sum(j in 1..9)(solution[i, j]) == 45 /\
    sum(j in 1..9)(solution[j, i]) == 45
);

% Suma redundante por Bloque 3x3
constraint forall(k, l in 0..2) (
    sum(i, j in 1..3)(solution[i + 3*k, j + 3*l]) == 45
);
```

Observe que estas restricciones redundantes permiten reducir el dominio de valores posibles que pueden ocupar las casillas, particularmente para aquellas en filas o columnas con casillas ya previamente llenadas. Por ejemplo, si en una fila únicamente faltan por llenar dos casillas, y la suma de los valores en la fila es igual a 28, entonces las dos casillas restantes deben sumar $45 - 28 = 17$, lo que permite reducir el dominio de las dos variables restantes al intervalo [8.,9], lo que permite podar enormemente el árbol de búsqueda.

Rompimiento de Simetrías

Observe que, dado que una instancia particular de Sudoku se establece por medio de unos valores iniciales fijos que no pueden modificarse. Para una misma instancia del puzzle, no se garantiza que existan soluciones simétricas. Esto se debe a que, aún cuando a partir de una solución válida se puedan obtener otros tableros resueltos por medio de rotaciones, reflexiones y permutaciones de columnas y filas, no se puede garantizar que estas nuevas soluciones sean también solución del problema de Sudoku original, pues es posible que las casillas originales se hayan desplazado. Por lo que serían soluciones válidas para instancias distintas del Puzzle.

Como ocurre por ejemplo en el Sudoku de la Figura 2, esa instancia al ser bien definida posee una única solución válida, aún cuando la rotación o reflexión de la solución resulte en tableros de Sudoku igualmente resueltos, no son solución al mismo problema.

Estas soluciones simétricas sólo son soluciones al problema original cuando la posición de las casillas iniciales sea también invariante bajo rotaciones o reflexiones, por ejemplo situándose en el centro del tablero. Como ocurre por ejemplo con la instancia en la Figura 3, observe que al reflejar verticalmente la solución de la Figura 3b se obtiene una solución igualmente válida al mismo problema.

Pero dado que esta condición no se puede garantizar de forma general, para este problema el rompimiento de simetrías no es necesario o útil.

También sería útil el rompimiento de simetrías si el problema consistiera de hallar la totalidad de tableros posibles de Sudokus, es decir, todas las soluciones posibles.

								8
1	8				2	3		
	6			5	7			1
	7		9	6				
	9		7		4		1	
				8	1		4	
6			2	4			8	
		4	5				9	3
5								

(a) Instancia bien definida del Sudoku

7	4	2	6	1	3	9	5	8
1	8	5	4	9	2	3	6	7
9	6	3	8	5	7	4	2	1
4	7	1	9	6	5	8	3	2
3	9	8	7	2	4	5	1	6
2	5	6	3	8	1	7	4	9
6	3	7	2	4	9	1	8	5
8	1	4	5	7	6	2	9	3
5	2	9	1	3	8	6	7	4

(b) Única solución válida de la instancia

Figura 2: Instancia bien definida del Sudoku y su única solución válida.

				1				
				2				
				3				
				4				
				5				
				6				
				7				
				8				
				9				

(a) Instancia con múltiples soluciones simétricas

2	3	6	4	1	5	7	8	9
8	9	5	6	2	7	3	4	1
7	4	1	8	3	9	2	6	5
6	2	8	3	4	1	9	5	7
4	1	9	7	5	2	8	3	6
3	5	7	9	6	8	1	2	4
9	8	2	5	7	4	6	1	3
5	7	3	1	8	6	4	9	2
1	6	4	2	9	3	5	7	8

(b) Una posible solución

Figura 3: Instancia del Sudoku con múltiples soluciones simétricas

1.3. Estrategias de Búsqueda y Distribución

Para el análisis de eficiencia de solución se implementaron las siguientes estrategias de búsqueda:

- *Búsqueda principiante:* Similar a cómo un principiante en la resolución de Sudoku podría aproximarse a la solución del Puzzle, se recorre el arreglo de solución de izquierda a derecha y de arriba hacia abajo (según el aplanamiento en 1D) y en cada casilla se prueban valores en el orden del dominio, empezando por el valor posible más pequeño. Esta estrategia no realiza ninguna decisión en particular para acelerar el proceso de búsqueda de solución. Para ello, se emplean las anotaciones `input_order` para la selección de variables y `indomain_min` para la selección de valores:

```
%Búsqueda principiante
solve :: int_search(
        solution,
        input_order,
        indomain_min,
        complete)
        satisfy;
```

.

- *Búsqueda intermedia:* Una táctica un poco más avanzada consiste en seleccionar las casillas con la menor cantidad de valores posibles, dado que probar valores incorrectos en estas casillas llevará más rápido a contradicciones, reduciendo así la cantidad de ramas que se deben explorar antes de hallar una solución. Aún así, para el valor de prueba siempre se selecciona el menor valor posible. Para ello, se implementan las anotaciones:

```
%Búsqueda intermedia
solve :: int_search(
        solution,
        first_fail,
        indomain_random,
        complete)
        satisfy;
```

- *Búsqueda avanzada:* Por último, como la técnica de búsqueda más informada, se emplea la heurística proporcionada por la anotación `dom_w_deg`, la cual selecciona la variable que maximiza la razón entre el tamaño del dominio y la cantidad de veces que la variable ha resultado en una búsqueda fallida [**?**]. Adicionalmente, el valor de prueba ahora se selecciona de forma aleatoria en aras de disminuir la posibilidad de caer en ramas profundas de búsqueda por la preferencia de ciertos valores por sobre otros:

```
%Búsqueda avanzada
solve :: int_search(
        solution,
        dom_w_deg,
        indomain_random,
        complete)
        satisfy;
```

1.4. Pruebas y Análisis Experimental

Posteriormente, se probaron las distintas instancias de implementación de restricciones, en conjunto con las distintas estrategias de búsqueda comparando la cantidad de nodos explorados, el tiempo de respuesta y la forma general de los árboles de búsqueda resultantes. Para la prueba se partió de la instancia de Sudoku *."l Escargot"* diseñada por el matemático Finlandés Arto Inkala para para ser particularmente complejo de resolver, conservando aún una única solución. A partir de esta instancia, se eliminaron dos pistas para aumentar sustancialmente la cantidad de soluciones posibles, arribando al siguiente Sudoku de prueba:

A partir de esta instancia se probó el desempeño de las instancias y estrategias de búsqueda para hallar la totalidad de 3630 soluciones posibles, empleando en todos los casos el solver Gecode 6.4.0.

Los resultados de las pruebas se condensaron en la Tabla 1, la cual muestra los distintos tiempo de ejecución, nodos explorados, fallidos y la máxima profundidad alcanzada por el árbol de búsqueda. Para cada par de instancia y estrategias de búsqueda se realizó un total de 10 ejecuciones y se consolidó el valor promedio de tiempos de ejecución, tomados a partir del valor `solveTime` arrojado por Minizinc como parte de las estadísticas de solución, junto con su desviación estándar. Esto, a fin de reducir la variación en tiempos de ejecución por *cold starts* o el orden interno del *scheduler* de procesos de la CPU.

Observe que dado que se hallaron la totalidad de las soluciones del Sudoku, se obligó al solver a recorrer todas las ramas posibles, por lo que la cantidad de nodos explorados y nodos fallidos es constante para una misma búsqueda con instancia y estrategia de búsqueda.

1					7		9	
	3			2				8
		9	6			5		
		5				9		
	1							2
6					4			
3							1	
	4							7
		7				3		

Figura 4: Versión mal definida de Sudoku *Al Escargot* empleada para pruebas de solución.

A partir de los resultados obtenidos, como evidencia la Tabla 1 el mejor par de instancia y solución en cuestión de número de nodos explorados y fallidos fue la instancia Base con la estrategia intermedia, explorando 9367 nodos y fallando en 1069 de ellos. Esto puede deberse a que la instancia base emplea la restricción global `all_different` la cual permite que los solvers como Gecode empleen técnicas de propagación especializadas que aceleran el proceso de búsqueda de solución, lo que no ocurren en las instancias alternativas en dónde las restricción de diferencia se implementan de forma manual, evidenciado por el aumento de nodos explorados para las instancias alternativas de restricciones del modelo.

Esta diferencia entre la instancia base y alternativa se hace más evidente al examinar los árboles de búsqueda. Como evidencia la Figura 5 obtenidos por medio de la búsqueda de la única solución del Sudoku *Al Escargot* con el solver Gecode Gist, se observa como la instancia base al implementar las restricciones con `all_different` permite la reducción del árbol de búsqueda visitando menos ramas fallidas al fallar más temprano resultante en un árbol menos profundo, con menos nodos y por lo tanto reduciendo los costos de búsqueda de la solución.

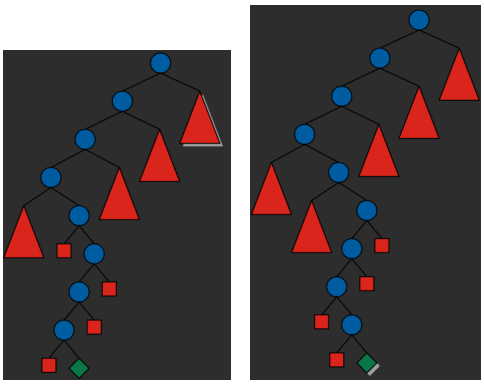
Por otro lado, la técnica intermedia exploró la menor cantidad de nodos y falló la menor cantidad de veces dado que aunque la técnica avanzada de selección de variables `dom_w_weg` puede aumentar enormemente la posibilidad del solver de fallar de forma temprana, este beneficio es mucho más evidente en problemas de mayor complejidad computacional y restricciones más estrictas que el Sudoku. Esto resulta en que el overhead creado por el periodo de *aprendizaje* que requiere la estrategia de búsqueda y la prioridad que le otorga a variables que ya hayan fallado puede resultar en la toma de decision menos óptimas que simplemente las variables de menor dominio, como realiza `first_fail`.

Adicionalmente, la estrategia de selección de valor `first_domain` es preferible en el caso del Sudoku por sobre `domain_random` dado que no genera un overhead de selección por generación de números aleatorios, lo cuál no vale la pena dada la naturaleza de por sí reducida del dominio de las variables en el Sudoku.

Aún así, observe que los tiempos de ejecución de todas las instancias y estrategias fueron sumamente similares, variando únicamente en unas pocas centésimas de segundo, lo que resulta de la pequeña magnitud del problema, que no permite que las diferencias en las estrategias de búsqueda se hagan evidentes por sobre las variaciones inherentes en tiempos de procesamiento innatos a variaciones en hardware y software.

Instancia	Estrategia	Tiempo (s)	Nodos explorados	Nodos fallidos	Profundidad Máxima
Base	Principiante	1.542746 ± 0.0438	9467	1104	18
Base	Intermedia	1.548447 ± 0.0741	9397	1069	18
Base	Avanzada	1.532463 ± 0.0783	9601	1171	22
Alt. con Redundancias	Principiante	1.601262 ± 0.0564	11653	2197	18
Alt. con Redundancias	Intermedia	1.569572 ± 0.0484	11289	2015	18
Alt. con Redundancias	Avanzada	1.525641 ± 0.0652	9469	1105	22
Alt. sin Redundancias	Principiante	1.49597 ± 0.0843	11653	2197	18
Alt. sin Redundancias	Intermedia	1.519158 ± 0.0655	11289	2015	18
Alt. sin Redundancias	Avanzada	1.507931 ± 0.0897	9695	1218	22

Tabla 1: Comparación de rendimiento para el problema del Sudoku.



(a) Instancia Base (b) Instancia alternativa sin redundancia

Figura 5: Árboles de búsqueda para las instancias Base y Alternativas sin Redundancia para la única solución del Sudoku Al Escargot, con estrategia de búsqueda intermedia.

1.5. Conclusiones del Ejercicio

A partir de la implementación de las distintas versiones de modelo y estrategias de búsqueda para la determinación de soluciones posibles para un Sudoku dado, se hizo evidente que la mejor aproximación al problema consiste en el empleo de restricciones globales ya optimizadas por el solver como `all_different` para la implementación de las distintas restricciones. Adicionalmente, se debe tener en cuenta la verdadera magnitud del problema, incluyendo el número total de variables y la amplitud de dominio de las dominios para seleccionar apropiadamente las estrategias de búsqueda que puedan alcanzar la mayor cantidad de mejorías teniendo en cuenta que aquellas heurísticas de mayor complejidad resultan igualmente en overheads de cálculo mayores, lo que, para problemas de magnitud relativamente pequeña como el Sudoku demostraron no ser

viables ni óptimos.

2. Problema 2: Kakuro

2.1. Descripción del Problema y Abstracción CSP

El Kakuro (también conocido como *Cross Sums*) es un rompecabezas numérico de lógica combinatoria equivalente a un crucigrama matemático. El tablero está conformado por celdas negras (no jugables, que actúan como separadores y contienen las pistas numéricas) y celdas blancas (jugables). Una sucesión continua de casillas blancas adyacentes en dirección horizontal o vertical constituye un *tramo* o *serie*.

El objetivo consiste en asignar un dígito entero del 1 al 9 a cada casilla blanca del tablero de modo que se satisfagan simultáneamente dos condiciones fundamentales en cada serie horizontal y vertical:

1. La suma aritmética de los dígitos asignados a las casillas de la serie debe ser exactamente igual al valor de la pista asociada a dicho tramo.

2. Ningún dígito puede repetirse dentro de una misma serie (horizontal o vertical), aunque un mismo dígito sí puede aparecer varias veces en una misma fila o columna si sus apariciones están separadas por al menos una casilla negra.

Desde la perspectiva de la Programación por Restricciones (CSP), el Kakuro se abstrae como un problema de satisfacción sobre dominios finitos enteros donde cada celda blanca corresponde a una variable de decisión restringida por la intersección de una restricción lineal de suma y una restricción de disyunción por pares (o restricción global de diferencia) en su tramo horizontal y en su tramo vertical.

2.1.1. Parámetros y Datos de Entrada

Siguiendo una representación matricial paramétrica que separa completamente el modelo (`kakuro.mzn`) de los datos de cada instancia (`.dzn`), se definen los siguientes parámetros de entrada:

- $F \in \mathbb{Z}^+$: Número de filas del área jugable del tablero.
- $C \in \mathbb{Z}^+$: Número de columnas del área jugable del tablero.
- $n_H \in \mathbb{Z}^+$: Número total de series (tramos) horizontales.
- $n_V \in \mathbb{Z}^+$: Número total de series (tramos) verticales.
- $P^H \in \mathbb{Z}^{n_H \times 4}$: Matriz de pistas horizontales (`pistas_h`), donde cada fila $k \in \{1, \ldots, n_H\}$ representa la tupla $(f_k, c_k^{\text{ini}}, c_k^{\text{fin}}, S_k^H)$, indicando que en la fila f_k el tramo horizontal va desde la columna c_k^{ini} hasta la columna c_k^{fin} y debe sumar S_k^H.
- $P^V \in \mathbb{Z}^{n_V \times 4}$: Matriz de pistas verticales (`pistas_v`), donde cada fila $m \in \{1, \ldots, n_V\}$ representa la tupla $(c_m, f_m^{\text{ini}}, f_m^{\text{fin}}, S_m^V)$, indicando que en la columna c_m el tramo vertical va desde la fila f_m^{ini} hasta la fila f_m^{fin} y debe sumar S_m^V.

A partir de las pistas horizontales se define estáticamente el conjunto de posiciones de casillas blancas $B \subseteq \{1, \ldots, F\} \times \{1, \ldots, C\}$ como:

$$B = \left\{ (i,j) \in \{1,\ldots,F\} \times \{1,\ldots,C\} \;\middle|\; \exists k \in \{1,\ldots,n_H\} : f_k = i \wedge c_k^{\text{ini}} \leq j \leq c_k^{\text{fin}} \right\}$$

2.1.2. Variables de Decisión y Dominios

Se define una matriz bidimensional de variables de decisión $X_{i,j}$ (`tablero[i,j]`) para cada celda $(i, j) \in \{1, \ldots, F\} \times \{1, \ldots, C\}$. Con el fin de establecer el dominio más pequeño posible para cada tipo de casilla sin recurrir a tipos opcionales:

- Para toda casilla blanca $(i, j) \in B$, la variable representa el dígito escrito en la celda y su dominio efectivo es:

$$D(X_{i,j}) = \{1, 2, 3, 4, 5, 6, 7, 8, 9\}, \quad \forall (i, j) \in B$$

- Para toda casilla negra $(i, j) \notin B$, la variable se fija mediante una restricción unaria al valor constante cero, reduciendo su dominio a un único elemento:

$$D(X_{i,j}) = \{0\}, \quad \forall (i, j) \notin B$$

2.1.3. Restricciones del Modelo

El modelo matemático principal se compone de las siguientes familias de restricciones:

1. **Partición de dominios entre casillas blancas y negras:**

$$\forall i \in \{1, \ldots, F\},\ \forall j \in \{1, \ldots, C\} : \quad \begin{cases} X_{i,j} \geq 1 & \text{si } (i, j) \in B \\ X_{i,j} = 0 & \text{si } (i, j) \notin B \end{cases}$$

2. **Suma exacta en cada serie horizontal:**

$$\forall k \in \{1, \ldots, n_H\} : \quad \sum_{j=c_k^{\text{ini}}}^{c_k^{\text{fin}}} X_{f_k,j} = S_k^H$$

3. **Suma exacta en cada serie vertical:**

$$\forall m \in \{1, \ldots, n_V\} : \quad \sum_{i=f_m^{\text{ini}}}^{f_m^{\text{fin}}} X_{i,c_m} = S_m^V$$

4. **No repetición de dígitos por tramo (Formulación A — Desigualdades binarias):**

$$\forall k \in \{1, \ldots, n_H\},\ \forall j_1, j_2 \in \{c_k^{\text{ini}}, \ldots, c_k^{\text{fin}}\} \text{ con } j_1 < j_2 : \quad X_{f_k,j_1} \neq X_{f_k,j_2}$$

$$\forall m \in \{1, \ldots, n_V\},\ \forall i_1, i_2 \in \{f_m^{\text{ini}}, \ldots, f_m^{\text{fin}}\} \text{ con } i_1 < i_2 : \quad X_{i_1,c_m} \neq X_{i_2,c_m}$$

5. **No repetición de dígitos por tramo (Formulación B — Restricción global `alldifferent`):**

$$\forall k \in \{1, \ldots, n_H\} : \quad \text{alldifferent}\left(\left[X_{f_k,j} \;\middle|\; j \in c_k^{\text{ini}} \ldots c_k^{\text{fin}}\right]\right)$$

$$\forall m \in \{1, \ldots, n_V\} : \quad \text{alldifferent}\left(\left[X_{i,c_m} \;\middle|\; i \in f_m^{\text{ini}} \ldots f_m^{\text{fin}}\right]\right)$$

2.2. Detalles de Implementación (`kakuro.mzn`)

La implementación en `kakuro.mzn` se estructuró de manera completamente paramétrica:

- **Validación temprana de datos (`assert`):** Antes de instanciar las restricciones, se verifica que las dimensiones F, C, n_H, n_V sean positivas y que la suma total de las pistas horizontales coincida exactamente con la suma total de las pistas verticales ($\sum_k S_k^H = \sum_m S_m^V$), previniendo errores de transcripción en los archivos `.dzn`.
- **Función booleana de pertenencia (`es_blanca`):** Se definió una función auxiliar determinista (`par bool`) que evalúa mediante un cuantificador existencial (`exists`) si la coordenada (i, j) pertenece a algún tramo horizontal. Esto permite tanto acotar los dominios del tablero como construir por comprensión el arreglo unidimensional `vars_blancas` que contiene exclusivamente las variables de decisión de las casillas blancas, evitando que las estrategias de búsqueda ramifiquen trivialmente sobre las casillas negras fijadas en 0.
- **Banderas de control experimental:** Se incorporaron los parámetros booleanos `usar_alldifferent` y `usar_redundantes` para activar o desactivar las variantes de modelado y evaluar su impacto real sobre el número de propagadores y nodos.

2.2.1. Variantes en el Modelado de Restricciones

Se implementaron y contrastaron dos enfoques para garantizar la unicidad de dígitos en cada tramo de longitud L:

1. **Descomposición en desigualdades binarias ($\neq$):** Genera $\binom{L}{2} = \frac{L(L-1)}{2}$ restricciones binarias independientes por cada serie. Su desventaja principal en propagación es su carácter estrictamente local: una desigualdad $X_a \neq X_b$ permanece inactiva hasta que una de las dos variables queda instanciada a un único valor (consistencia de arco débil sobre $\neq$), por lo que no detecta conflictos de cardinalidad cuando varias variables comparten un subconjunto reducido de valores.
2. **Restricción global `alldifferent`:** Agrupa todas las variables de la serie en un único propagador global por tramo, reduciendo sustancialmente el número de propagadores en el grafo de restricciones y permitiendo detectar subconjuntos saturados de Hall antes de que las variables estén fijas.

2.2.2. Restricciones Redundantes y Rompimiento de Simetrías

Restricciones Redundantes. En el modelo básico, el propagador de la suma lineal $\sum_{r=1}^{L} X_r = S$ desconoce que las variables X_r deben tomar valores distintos entre sí; por ello, calcula sus cotas asumiendo que las $L-1$ variables restantes podrían repetir el valor mínimo 1 o el valor máximo 9. Para acoplar semánticamente la restricción de suma con la restricción `alldifferent`, se dedujeron e implementaron dos familias de restricciones redundantes:

1. **Cotas combinatorias extremas por longitud de serie:** En un tramo de longitud $L = \text{fin} - \text{ini} + 1$ y suma objetivo S, las otras $L-1$ casillas son dígitos distintos en $\{1, \ldots, 9\}$. Por tanto, la suma mínima posible de las otras $L-1$ casillas es $\sum_{r=1}^{L-1} r = \frac{(L-1)L}{2}$, y su suma máxima posible es $\sum_{r=11-L}^{9} r = \frac{(L-1)(20-L)}{2}$. En consecuencia, toda variable X perteneciente a dicho tramo debe satisfacer obligatoriamente:

$$S - \frac{(L-1)(20-L)}{2} \leq X \leq S - \frac{(L-1)L}{2}$$

Por ejemplo, en una serie de longitud $L = 2$ con suma $S = 4$, esta cota deduce de inmediato en el nodo raíz que $X \leq 4 - 1 = 3$, eliminando los valores $\{4, 5, 6, 7, 8, 9\}$ antes de iniciar

la búsqueda. En el preprocesamiento de MiniZinc, esta poda de dominios permite incluso fijar variables de manera directa y simplificar propagadores enteros.

2. **Invariante de suma global del tablero:** Dado que las series horizontales particionan el conjunto de casillas blancas B, la suma de todas las celdas del tablero debe coincidir con la suma acumulada de todas las pistas horizontales:

$$\sum_{i=1}^{F}\sum_{j=1}^{C} X_{i,j} = \sum_{k=1}^{n_H} S_k^H$$

Análisis de Simetrías. A diferencia de problemas de empaquetamiento o asignación con objetos indistinguibles, el problema de Kakuro **no presenta simetrías de variables ni de valores**. Cada celda blanca $(i, j) \in B$ está identificada de forma única por la intersección de una serie horizontal específica (con suma S_k^H) y una serie vertical específica (con suma S_m^V). Si se permutaran los valores de dos casillas dentro de un mismo tramo horizontal, se alterarían las sumas de los tramos verticales ortogonales que se cruzan en dichas columnas. Asimismo, en un tablero de Kakuro bien propuesto la solución es única, por lo que no existen soluciones simétricas equivalentes ni se requieren restricciones de rompimiento de simetrías.

2.3. Estrategias de Búsqueda y Distribución

Sobre el arreglo unidimensional `vars_blancas` (que filtra únicamente las variables con $(i, j) \in B$) se evaluaron seis configuraciones de búsqueda mediante `int_search`:

1. **Default (`solve satisfy`):** Estrategia nativa por defecto del solver Gecode.
2. **`input_order` + `indomain_min`:** Selecciona las casillas blancas en orden lexicográfico de lectura (fila por fila, de izquierda a derecha) y prueba primero el menor dígito disponible.
3. **`input_order` + `indomain_max`:** Selecciona las casillas en orden de fila pero asigna primero el mayor dígito disponible en el dominio.
4. **`first_fail` + `indomain_min`:** Selecciona dinámicamente la casilla blanca con el dominio actual más pequeño (principio de fallo primero), priorizando las intersecciones de tramos cortos o sumas extremas, y prueba el menor valor disponible.
5. **`first_fail` + `indomain_split`:** Combina la selección de variable más restringida con la bisección de su dominio ($\leq m$ vs. $> m$).
6. **`dom_w_deg` + `indomain_min`:** Selecciona la variable según la razón entre el tamaño de su dominio y su grado ponderado de fallos acumulados en los propagadores asociados.

2.4. Pruebas y Análisis Experimental

Se construyó una batería de tres instancias en archivos de datos independientes:

- `kakuro1.dzn` **(Tablero 1 del taller):** Área jugable de 5×4 con 14 casillas blancas, 5 series horizontales y 4 series verticales (suma total = 65).
- `kakuro2.dzn` **(Tablero 2 del taller):** Área jugable de 7×7 con 24 casillas blancas dispuestas en anillo diagonal, 9 series horizontales y 9 series verticales (suma total = 140).
- `kakuro3.dzn` **(Tablero 3 adicional de mayor escala):** Área jugable de 8×8 con 36 casillas blancas distribuidas en bandas diagonales, 16 series horizontales y 16 series verticales (suma total = 205).

Las tres instancias poseen solución única verificada mediante búsqueda exhaustiva (`--all-solutions`). Para los dos tableros del enunciado del taller, las matrices solución obtenidas por el modelo son:

$$\text{Kakuro 1 } (5\times 4) = \begin{pmatrix} 2 & 6 & 1 & \cdot \\ 1 & 9 & 3 & \cdot \\ 6 & 7 & \cdot & \cdot \\ \cdot & 4 & 2 & 1 \\ \cdot & 8 & 9 & 2 \end{pmatrix}, \qquad \text{Kakuro 2 } (7\times 7) = \begin{pmatrix} \cdot & \cdot & 2 & 7 & \cdot & \cdot & \cdot \\ \cdot & \cdot & 7 & 8 & 9 & \cdot & \cdot \\ 2 & 4 & 1 & \cdot & 4 & 8 & \cdot \\ 5 & 7 & \cdot & \cdot & \cdot & 9 & 1 \\ \cdot & 9 & 5 & \cdot & 1 & 6 & 2 \\ \cdot & \cdot & 9 & 8 & 5 & \cdot & \cdot \\ \cdot & \cdot & \cdot & 9 & 2 & \cdot & \cdot \end{pmatrix}$$

En el Cuadro 2 se presenta el estudio de ablación de las tres variantes de modelado al explorar la totalidad del espacio de búsqueda (`--all-solutions` con estrategia `first_fail + indomain_min`) para encontrar la solución y demostrar su unicidad. Se observa con claridad cómo el paso de desigualdades binarias ($\neq$) a `alldifferent` reduce el número de propagadores entre un 28 % y un 49 %, mientras que la incorporación de las cotas combinatorias redundantes poda los dominios desde la etapa de aplanamiento, reduciendo los propagadores activos hasta en un 87,5 % (de 104 a 13 en `kakuro3.dzn`) y disminuyendo las propagaciones y nodos en más de la mitad.

Instancia	Variante de Modelado	Propagadores	Propagaciones	Nodos	Fallos	Tiempo (ms)
`kakuro1.dzn`	A: Desigualdades ($\neq$) sin red.	39	151	9	4	0.13
(5×4, 14 blancas)	B: `alldifferent` sin red.	20	134	9	4	0.14
	C: `alldifferent` + Redundantes	**13**	**57**	**5**	**2**	**0.13**
`kakuro2.dzn`	A: Desigualdades ($\neq$) sin red.	56	164	9	4	0.16
(7×7, 24 blancas)	B: `alldifferent` sin red.	40	158	9	4	0.15
	C: `alldifferent` + Redundantes	**9**	**62**	**3**	**1**	**0.12**
`kakuro3.dzn`	A: Desigualdades ($\neq$) sin red.	104	336	11	5	0.22
(8×8, 36 blancas)	B: `alldifferent` sin red.	64	305	11	5	0.22
	C: `alldifferent` + Redundantes	**13**	**114**	**3**	**1**	**0.15**

Tabla 2: Comparación experimental de las variantes de modelado en Kakuro (búsqueda exhaustiva `--all-solutions`).

Por su parte, el Cuadro 3 compara el desempeño de las seis estrategias de búsqueda evaluadas para hallar la primera solución tanto sobre el modelo sin restricciones redundantes (Variante B) como sobre el modelo completo con restricciones redundantes (Variante C).

2.5. Conclusiones del Ejercicio

- **Sinergia entre restricciones globales y cotas redundantes:** Aunque reemplazar las desigualdades binarias ($\neq$) por la restricción global `alldifferent` reduce drásticamente el número de propagadores del modelo, el mayor salto de eficiencia se obtiene al añadir las cotas combinatorias redundantes $S - \frac{(L-1)(20-L)}{2} \leq X \leq S - \frac{(L-1)L}{2}$. Estas desigualdades inyectan el conocimiento de no repetición directamente sobre los límites de cada variable antes de iniciar la búsqueda, resolviendo `kakuro2.dzn` y `kakuro3.dzn` prácticamente en el nodo raíz (2 nodos y 0 fallos).
- **Comportamiento de las heurísticas de búsqueda:** Sin restricciones redundantes, en `kakuro1.dzn` las heurísticas dinámicas (`first_fail`) superan al orden estático (`input_order`) y a la búsqueda por defecto al elegir primero las casillas de tramos cortos (longitud 2), reduciendo la profundidad máxima del árbol de 5 a 3. Con las restricciones redundantes activas, los dominios iniciales quedan tan acotados que tanto `first_fail + indomain_min` como `first_fail + indomain_split` convergen de forma inmediata con una profundidad máxima de apenas 1 nivel.

Instancia	Estrategia de Búsqueda	Sin Redundantes (Variante B)			Con Redundantes (Variante C)		
		Tiempo (ms)	Nodos	Fallos	Tiempo (ms)	Nodos	Fallos
`kakuro1.dzn`	Default (`solve satisfy`)	0.12	10	4	0.17	4	1
	`input_order + indomain_min`	0.12	8	3	0.11	6	2
	`input_order + indomain_max`	0.14	7	2	0.13	4	1
	`first_fail + indomain_min`	0.12	7	2	0.15	5	2
	`first_fail + indomain_split`	0.12	7	2	**0.10**	5	2
	`dom_w_deg + indomain_min`	0.12	8	3	0.11	6	2
`kakuro2.dzn`	Default (`solve satisfy`)	0.13	4	0	0.14	2	0
	`input_order + indomain_min`	0.13	5	1	0.18	2	0
	`input_order + indomain_max`	0.14	8	3	0.12	3	1
	`first_fail + indomain_min`	0.15	8	3	**0.11**	**2**	**0**
	`first_fail + indomain_split`	0.21	8	3	0.16	2	0
	`dom_w_deg + indomain_min`	0.22	8	3	0.15	2	0
`kakuro3.dzn`	Default (`solve satisfy`)	0.18	7	1	0.21	3	0
	`input_order + indomain_min`	0.19	7	1	**0.16**	**2**	**0**
	`input_order + indomain_max`	0.22	10	4	0.24	3	1
	`first_fail + indomain_min`	0.36	7	1	0.24	**2**	**0**
	`first_fail + indomain_split`	0.19	7	1	0.19	**2**	**0**
	`dom_w_deg + indomain_min`	0.29	7	1	0.16	**2**	**0**

Tabla 3: Comparación experimental de estrategias de búsqueda en el modelo de Kakuro.

3. Problema 3: Secuencia Mágica

3.1. Descripción del Problema y Abstracción CSP

Una *secuencia mágica* (también denominada secuencia autodescriptiva) de longitud $n \in \mathbb{Z}^+$ es una tupla ordenada de números enteros $S = (x_0, x_1, \ldots, x_{n-1})$ tal que cada componente x_i pertenece al conjunto $\{0, 1, \ldots, n-1\}$ y cuantifica exactamente el número de apariciones del entero i dentro de la propia secuencia S. El objetivo computacional consiste en enumerar exhaustivamente todas las secuencias mágicas existentes para una longitud arbitraria n dada como entrada.

El paradigma de Programación por Restricciones sobre dominios finitos (CSP($\mathcal{FD}$)) resulta idóneo para este problema debido a su naturaleza autorreferencial y combinatoria: cada variable desempeña simultáneamente el rol de *valor posicionado* en la secuencia y de *cardinalidad objetivo* para uno de los elementos del universo. Mientras que una búsqueda por fuerza bruta exigiría evaluar n^n asignaciones candidatas (10^{10} en $n = 10$ y 10^{200} en $n = 100$), un motor de propagación sobre dominios finitos explota las relaciones globales de conteo y las leyes algebraicas invariantes de la secuencia para descartar subespacios infactibles antes de instanciar las variables.

Siguiendo el formalismo de Apt adoptado en el curso, para cada instancia parametrizada por n se define el problema de satisfacción de restricciones $\mathcal{P}_{\mathrm{SM}}(n) := \langle \mathcal{C} \ ; \ \mathcal{DE} \rangle$ cuyos elementos constitutivos se precisan a continuación.

3.1.1. Parámetros y Datos de Entrada

El modelo depende de un único parámetro escalar externo, suministrado mediante archivos de datos independientes:

- $n \in \mathbb{Z}^+$ ($n \geq 1$): entero positivo que especifica la longitud de la secuencia mágica buscada.
- $I = \{0, 1, \ldots, n-1\}$: conjunto finito de índices (y simultáneamente universo de valores admisibles) derivado determinísticamente a partir del parámetro n.

Ningún otro dato es requerido en la entrada, garantizando una separación estricta entre los parámetros de instancia y las incógnitas del modelo.

3.1.2. Variables de Decisión y Dominios

El conjunto de variables de decisión está conformado por n variables enteras indexadas sobre I:

$$X = \{x_0, x_1, \ldots, x_{n-1}\}$$

donde cada variable x_i representa el entero ubicado en la posición i-ésima de la secuencia S y, de forma equivalente por definición del problema, el número total de veces que el valor i ocurre en S.

La declaración de dominios finitos $\mathcal{DE}$ asigna inicialmente a cada variable el intervalo entero acotado:

$$\mathcal{DE} = \left\{ x_i \in D(x_i) \;\middle|\; i \in I \right\}, \qquad D(x_i) = \{0, 1, \ldots, n-1\} \quad \forall i \in I.$$

Este dominio excluye *a priori* todos los valores semánticamente imposibles sin perder soluciones válidas:

1. **Exclusión de enteros negativos ($x_i < 0$):** dado que x_i denota la cardinalidad de un subconjunto de posiciones de I, necesariamente $x_i \geq 0$.

2. **Exclusión de valores mayores o iguales a n ($x_i \geq n$):** como la secuencia posee exactamente n posiciones, ningún valor puede aparecer más de n veces ($x_i \leq n$). Más aún, el valor extremo $x_i = n$ también es imposible para todo $i \in I$: si existiese algún $i \in I$ con $x_i = n$, las n componentes de la secuencia tendrían que ser iguales a i; en particular, se tendría $x_i = i$, lo que implicaría $i = n \notin \{0, \ldots, n-1\}$, llegando a una contradicción directa. Por consiguiente, la cota superior estricta $n - 1$ es exacta *a priori*.

3.1.3. Restricciones Básicas

Sea $[P] \in \{0, 1\}$ la notación de corchete de Iverson, que evalúa a 1 si el predicado lógico P es verdadero y a 0 en caso contrario. El conjunto de restricciones primarias $\mathcal{C}_{\text{base}}$ formaliza de manera incondicional la propiedad autodescriptiva para cada índice $i \in I$:

$$C_i : \quad \sum_{j=0}^{n-1} [x_j = i] = x_i, \qquad \forall i \in I. \tag{1}$$

De forma equivalente en términos relacionales sobre el vector $X = (x_0, \ldots, x_{n-1})$, cada restricción C_i impone la relación global $\text{count}(X, i, x_i)$, exigiendo que la cardinalidad del conjunto de índices $\{j \in I \mid x_j = i\}$ coincida exactamente con el valor tomado por la variable x_i. Nótese que todas las restricciones de $\mathcal{C}_{\text{base}}$ se formulan directamente sobre las variables de decisión y los índices del parámetro I, sin condicionar la activación de ninguna restricción al valor particular asumido por una variable.

3.2. Detalles de Implementación (`secuencia.mzn`)

El modelo implementado en la ruta `problemas/2.3/secuencia.mzn` recibe el parámetro n, verifica su validez de dominio mediante una aserción de entrada ($n \geq 1$), declara el arreglo de variables de decisión indexado sobre $\{0, \ldots, n-1\}$ con dominio finito $\{0, \ldots, n-1\}$ e importa la biblioteca estándar de restricciones globales de MiniZinc. Para materializar el sistema de restricciones (1), se estudiaron y compararon experimentalmente tres codificaciones declarativas equivalentes en el plano semántico pero distintas en su traducción a FlatZinc:

1. **Suma explícita de igualdades reificadas:** traduce literalmente la ecuación (1) introduciendo n^2 variables booleanas auxiliares $b_{ij} \iff (x_j = i)$ y n ecuaciones lineales $\sum_{j=0}^{n-1} b_{ij} = x_i$. Aunque es conceptualmente transparente, genera una sobrecarga cuadrática de $\Theta(n^2)$ variables y propagadores reificados.

2. **Restricción global de conteo por valor (`count`):** impone la relación global de conteo para cada $i \in I$ vinculando el arreglo completo X, la constante i y la variable contadora x_i. En Gecode, esta formulación evita crear variables auxiliares enteras adicionales en FlatZinc (manteniendo exactamente n variables enteras y n propagadores globales de conteo), logrando un equilibrio sobresaliente entre legibilidad estructural y costo de propagación. Esta es la codificación adoptada en `secuencia.mzn`.

3. **Cardinalidad global cerrada (`global_cardinality_closed`):** agrupa las n condiciones de conteo en un único propagador global sobre el arreglo X, la lista cerrada de valores $\{0, \ldots, n-1\}$ y el propio arreglo X como vector de frecuencias. Esta variante concentra la consistencia de flujo en un solo propagador global de alto costo unitario pero mínimo número de invocaciones.

3.2.1. Restricciones Redundantes y Demostración Teórica

Siguiendo los lineamientos de la Sección 2.3 del taller, se incorporan al conjunto $\mathcal{C}$ dos restricciones lineales diofánticas redundantes:

$$R_1: \quad \sum_{i=0}^{n-1} x_i = n, \tag{2}$$

$$R_2: \quad \sum_{i=0}^{n-1} (i-1)x_i = 0 \quad \Longleftrightarrow \quad (-1)x_0 + 0 \cdot x_1 + 1 \cdot x_2 + \cdots + (n-2)x_{n-1} = 0. \tag{3}$$

Demostración de redundancia semántica. Sea $X = (x_0, \ldots, x_{n-1}) \in \{0, \ldots, n-1\}^n$ cualquier asignación que satisface el conjunto de restricciones básicas $\mathcal{C}_{\text{base}}$ dado en (1). Para cada valor $i \in I$, defínase el conjunto de posiciones donde aparece i como $P_i = \{j \in I \mid x_j = i\}$. Por (1), se cumple $|P_i| = x_i$ para todo $i \in I$.

- **Prueba de R_1:** Dado que cada componente x_j toma un único valor dentro de $I = \{0, \ldots, n-1\}$, la familia $\{P_i\}_{i=0}^{n-1}$ constituye una partición disjunta del conjunto de índices I ($P_i \cap P_k = \emptyset$ para $i \neq k$ y $\bigcup_{i=0}^{n-1} P_i = I$). Aplicando el principio de adición de cardinalidades sobre conjuntos disjuntos finitos:

$$\sum_{i=0}^{n-1} x_i = \sum_{i=0}^{n-1} |P_i| = \left| \bigsqcup_{i=0}^{n-1} P_i \right| = |I| = n.$$

  Por tanto, toda solución de $\mathcal{C}_{\text{base}}$ satisface necesariamente R_1.

- **Prueba de R_2:** Considérese la suma total de las entradas de la secuencia $\sum_{j=0}^{n-1} x_j$ evaluada por doble conteo agrupando los términos según la partición $\{P_i\}_{i=0}^{n-1}$. En cada subconjunto P_i, las $|P_i| = x_i$ componentes valen idénticamente i, de modo que:

$$\sum_{j=0}^{n-1} x_j = \sum_{i=0}^{n-1} \sum_{j \in P_i} x_j = \sum_{i=0}^{n-1} i \cdot |P_i| = \sum_{i=0}^{n-1} i \cdot x_i.$$

  Reemplazando el índice mudo j por i en el miembro izquierdo y transponiendo términos se obtiene:

$$\sum_{i=0}^{n-1} i \cdot x_i - \sum_{i=0}^{n-1} x_i = 0 \quad \Longleftrightarrow \quad \sum_{i=0}^{n-1} (i-1)x_i = 0.$$

  En consecuencia, $\mathcal{C}_{\text{base}} \models R_1 \wedge R_2$; añadir R_1 y R_2 preserva intacto el conjunto de soluciones $\text{Sol}(\mathcal{P}_{\text{SM}}(n))$.

Mecanismo operativo de poda de dominios. Aunque R_1 y R_2 son lógicamente redundantes, *no* son operacionalmente redundantes frente a la propagación local. Los propagadores individuales de conteo C_i operan de forma miope: mientras las variables x_j permanecen sin instanciar con dominio $\{0, \ldots, n-1\}$, cada C_i admite cualquier frecuencia $x_i \in \{0, \ldots, n-1\}$ porque todas las posiciones contienen aún a i en su dominio. En contraste, el propagador lineal de consistencia de cotas sobre R_2 reescribe la identidad como:

$$x_0 = \sum_{i=2}^{n-1} (i-1)x_i. \tag{4}$$

Como $(i-1) \geq 1$ y $x_i \geq 0$ para todo $i \geq 2$, y sabiendo que $\max D(x_0) = n-1$, en el propio **nodo raíz** (antes de cualquier ramificación) la consistencia de cotas fuerza para cada $i \geq 2$:

$$(i-1)x_i \leq x_0 \leq n-1 \quad \Longrightarrow \quad x_i \leq \left\lfloor \frac{n-1}{i-1} \right\rfloor.$$

Esta poda inicial colapsa dramáticamente el espacio cartesiano de búsqueda: para toda la segunda mitad de la secuencia ($i > \frac{n+1}{2}$), el cociente cumple $\lfloor \frac{n-1}{i-1} \rfloor = 1$, reduciendo instantáneamente el dominio de aproximadamente $n/2$ variables de tamaño n al dominio binario $\{0, 1\}$. Adicionalmente, al sumar R_1 y R_2 se obtiene $\sum_{i=0}^{n-1} ix_i = n$, acoplando globalmente la suma de frecuencias con el peso posicional. Por ejemplo, en $n = 3$, R_2 fija $x_0 = x_2 \leq 2$ y R_1 exige $2x_2 + x_1 = 3$; la paridad fuerza $x_1 \in \{1, 3\} \cap \{0, 1, 2\} = \{1\}$, de donde $x_0 = x_2 = 1$, contradiciendo de inmediato C_1 (pues el valor 1 aparecería tres veces pero $x_1 = 1$). Gracias a esta sinergia, el modelo con $R_1 + R_2$ refuta $n = 3$ en la raíz con 0 **nodos de decisión**, mientras que usar únicamente R_2 requiere 3 nodos y el modelo sin redundantes explora 5 nodos.

3.2.2. Rompimiento de Simetrías

Un análisis formal del grupo de automorfismos del problema demuestra que **no existen simetrías de variables ni de valores** en $\mathcal{P}_{\mathrm{SM}}(n)$, y que imponer restricciones canónicas de rompimiento de simetrías sería semánticamente incorrecto:

1. **Ausencia de simetrías de índice (variables):** Sea $\pi \in S_n \setminus \{\mathrm{id}\}$ una permutación no trivial de los índices I. Al permutar las posiciones de una secuencia X para formar X^π con $X_i^\pi = x_{\pi(i)}$, el multiconjunto de valores presentes en la secuencia permanece invariante: el número de apariciones de cada entero i en X^π sigue siendo x_i, pero la componente ubicada en la posición i pasa a ser $x_{\pi(i)}$. Para que X^π siga siendo mágica debería cumplirse $x_{\pi(i)} = x_i$ para todo $i \in I$. En cualquier secuencia mágica válida, las componentes no son constantes (por ejemplo, en $n = 4$, al permutar los dos primeros índices de $(1, 2, 1, 0)$ se obtiene $(2, 1, 1, 0)$, la cual viola C_0 porque el 0 aparece una sola vez mientras $x_0 = 2$).

2. **Ausencia de simetrías de valor:** Una biyección no trivial $\sigma \in S_n \setminus \{\mathrm{id}\}$ aplicada a los valores altera la suma $\sum_{i=0}^{n-1} \sigma(x_i) \neq n$ y rompe el acoplamiento autorreferencial entre el valor $\sigma(x_i)$ y la frecuencia del índice fijo i.

3. **Invalidez de restricciones de ordenamiento:** Imponer restricciones habituales de rompimiento de simetrías tales como orden creciente o decreciente ($x_0 \leq x_1 \leq \cdots \leq x_{n-1}$ o $x_0 \geq x_1 \geq \cdots \geq x_{n-1}$) eliminaría soluciones legítimas (y de hecho volvería insatisfacible el problema para $n \geq 5$, donde la única solución verifica $x_0 = n-4 > x_1 = 2 > x_2 = 1 > x_3 = 0 < x_{n-4} = 1 > x_{n-1} = 0$). Asimismo, en $n = 4$ las dos soluciones $(1, 2, 1, 0)$ y $(2, 0, 2, 0)$ poseen multiconjuntos de valores distintos ($\{0, 1, 1, 2\}$ frente a $\{0, 0, 2, 2\}$), por lo que son soluciones estructuralmente independientes que deben ser enumeradas en su totalidad.

3.3. Estrategias de Búsqueda y Distribución

En MiniZinc, una estrategia de distribución completa para dominios enteros divide recursivamente cada nodo abierto mediante ramificación binaria estándar de la forma $x_i = v$ en la rama izquierda frente a $x_i \neq v$ en la rama derecha (o $x_i \leq m$ frente a $x_i > m$ bajo bisección de dominio), ejecutando propagación hasta punto fijo tras cada decisión. Para determinar la estrategia óptima de enumeración en `secuencia.mzn`, se evaluaron nueve configuraciones combinando cinco heurísticas de selección de variable (`input_order`, orden inverso $x_{n-1} \dots x_0$, `first_fail`, `smallest` y `dom_w_deg`, además de la estrategia por defecto de Gecode) con tres heurísticas de selección de valor (`indomain_min`, `indomain_max` e `indomain_split`).

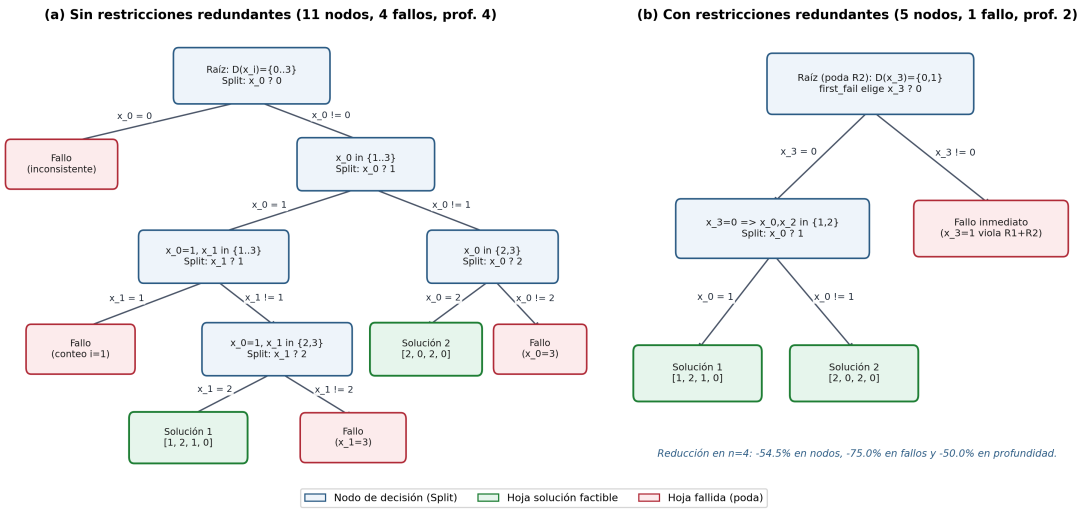


Figura 6: Árboles binarios de búsqueda completos trazados en Gecode para $n = 4$ bajo `first_fail` + `indomain_min`: (a) modelo base sin restricciones redundantes (11 nodos, 4 fallos, profundidad 4); (b) modelo con $R_1 + R_2$ (5 nodos, 1 fallo, profundidad 2).

La Figura 6 ilustra el efecto estructural de las restricciones redundantes sobre el árbol de búsqueda en $n = 4$:

- **Sin restricciones redundantes (Figura 6a):** En la raíz todos los dominios son uniformes $D(x_i) = \{0, 1, 2, 3\}$ de tamaño 4. Por regla de desempate posicional, `first_fail` degenera en `input_order` y selecciona x_0. La rama $x_0 = 0$ falla por contradicción en C_0, obligando a desplegar una cadena secuencial de decisiones $x_0 = 1$, $x_1 = 1$ (fallo), $x_1 = 2$ (Solución 1: $[1, 2, 1, 0]$), $x_1 = 3$ (fallo), $x_0 = 2$ (Solución 2: $[2, 0, 2, 0]$) y $x_0 = 3$ (fallo), acumulando 11 nodos, 4 hojas fallidas y 50 propagaciones.
- **Con restricciones redundantes $R_1 + R_2$ (Figura 6b):** En la raíz, la desigualdad (4) deduce $2x_3 + x_2 = x_0 \leq 3$, podando los dominios iniciales a $D(x_3) = \{0, 1\}$ (tamaño 2) y $D(x_2) = \{0, 1, 2, 3\} \cap [0, 3]$. Ahora `first_fail` detecta inmediatamente la variable más restringida x_3: en la rama derecha ($x_3 \neq 0 \Rightarrow x_3 = 1$), R_2 fuerza $x_0 \geq 2$ y $R_1 + C_3$ colapsan en fallo inmediato sin subramificaciones; en la rama izquierda ($x_3 = 0$), la propagación conjunta reduce $x_0, x_2 \in \{1, 2\}$, bastando una única decisión adicional ($x_0 = 1$ frente a $x_0 \neq 1$) para obtener directamente ambas soluciones válidas en solo 5 nodos, 1 fallo y 37 propagaciones. En la instancia insatisfacible $n = 6$, el mismo fenómeno reduce el árbol de 29 nodos y 15 fallos a apenas 7 nodos y 4 fallos.

Un hallazgo teórico-experimental fundamental, evidenciado en la Figura 7, es la **inversión del ranking entre heurísticas** producida por las restricciones redundantes:

- **Sin redundantes (M_0):** Al comenzar todas las variables con $|D(x_i)| = n$, `input_order` + `indomain_min` supera ampliamente a `first_fail` + `indomain_min` (55 frente a 117 nodos

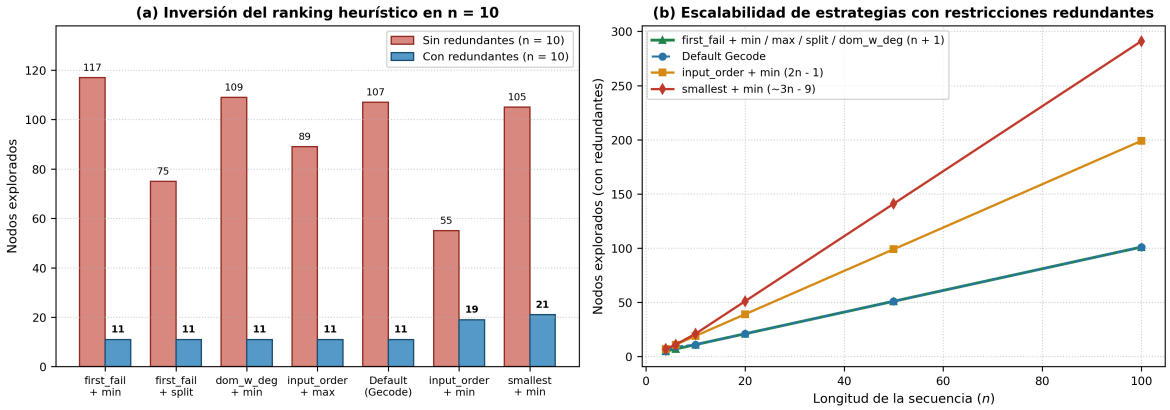


Figura 7: Interacción entre modelado y heurísticas de distribución en Secuencia Mágica: (a) inversión del ranking heurístico en $n = 10$ al incorporar $R_1 + R_2$; (b) escalabilidad lineal del número de nodos explorados en función de n para las estrategias evaluadas.

en $n = 10$; 775 frente a 10 557 nodos en $n = 100$), porque instanciar secuencialmente x_0 fija el número de ceros de la secuencia, mientras que `first_fail` salta dinámicamente hacia variables intermedias cuyos conteos aún no están acotados globalmente.

- **Con redundantes ($M_3 = \mathcal{C}_{\text{base}} \cup \{R_1, R_2\}$):** La poda en la raíz inducida por R_2 rompe la uniformidad inicial y asigna dominio binario $\{0, 1\}$ a todas las variables con $i > \frac{n+1}{2}$. Bajo esta geometría de dominios, `first_fail` + `indomain_min` (adoptada como estrategia oficial en `secuencia.mzn`) selecciona primero dichas variables de cola y prueba $x_i = 0$ (valor que coincide con la solución única para todo $i \neq n - 4$ cuando $n \geq 7$) y refuta en un solo paso la alternativa $x_i = 1$ gracias a R_2. Esto produce una ley cerrada óptima de $n + 1$ **nodos explorados y** $n/2$ **fallos para todo** $n \geq 8$ **par** (y, en general, $2\lceil n/2 \rceil + 1$ nodos y $\lceil n/2 \rceil$ fallos para todo $n \geq 12$, además de n nodos para $n \in \{7, 9, 11\}$), superando a `input_order` + `indomain_min` ($2n - 1$ nodos en n par), a `smallest` + `indomain_min` ($3n - 9$ nodos, penalizada por priorizar reiteradamente variables con mínimo 0 pero gran tamaño de dominio como x_0) e incluso superando a la búsqueda por defecto de Gecode y al orden inverso en $n = 6$ (7 frente a 9 nodos).

3.4. Pruebas y Análisis Experimental

Se diseñó una batería sistemática de 12 archivos de datos (`datos/2.3/secuencia_n*.dzn`) verificada automáticamente mediante el script de regresión `pruebas/2.3/test_secuencia.py` (que incluye además una prueba negativa de rechazo por aserción para entradas inválidas $n < 1$), cubriendo exhaustivamente todos los regímenes matemáticos del problema:

1. **Casos borde pequeños insatisfacibles ($n \in \{1, 2, 3\}$):** verifican la ausencia de falsos positivos y la detección temprana de inconsistencia sin ramificación (0 soluciones: en $n \in \{1, 2\}$ la contradicción se detecta estáticamente durante el aplanamiento a FlatZinc y en $n = 3$ en la propagación raíz).

2. **Caso multimodal ($n = 4$):** única longitud entera que admite exactamente 2 secuencias mágicas distintas: $(1, 2, 1, 0)$ y $(2, 0, 2, 0)$.

3. **Caso esporádico ($n = 5$) e insatisfacibilidad no trivial ($n = 6$):** validan la solución única $(2, 1, 2, 0, 0)$ en $n = 5$ y la refutación completa (0 soluciones) en el último entero positivo carente de secuencia mágica ($n = 6$).

4. **Familia paramétrica general y pruebas de estrés ($n \in \{7, 10, 15, 20, 50, 100\}$):** comprueban la convergencia hacia la ley estructural única para todo $n \geq 7$, dada por $x_0 = n-4$, $x_1 = 2$, $x_2 = 1$, $x_{n-4} = 1$ y $x_k = 0$ para todo $k \notin \{0, 1, 2, n-4\}$, así como la escalabilidad asintótica en instancias pares e impares.

Todas las instancias fueron ejecutadas con enumeración completa (`-a`) tanto en **Gecode 6.4.0** (propagación por dominios finitos) como en **Chuffed 0.14.0** (generación perezosa de cláusulas), verificando programáticamente coincidencia exacta en el conjunto de soluciones reportadas y el cumplimiento de la definición autodescriptiva en los 12 casos (Tabla 4). En $n = 100$, Gecode resuelve el modelo completo con 100 variables enteras, 101 nodos, 50 fallos y 14 711 propagaciones (1,43 ms de tiempo de resolución), mientras que Chuffed introduce 30 100 variables booleanas internas para codificar los literales de dominio, requiriendo 166 nodos, 55 fallos (55 *nogoods*) y 9,0 ms.

Archivo `.dzn`	Régimen	Sol.	Secuencia(s) enumerada(s)	Gecode 6.4.0			Chuffed 0.14.0	
(`datos/2.3/`)	evaluado	(`-a`)	($S = (x_0, \ldots, x_{n-1})$)	Nod.	Fall.	Propag.	Nod.	Fall.
`secuencia_n1`	Borde	0	`UNSAT` (en aplanamiento)	0	0	0	0	0
`secuencia_n2`	Borde	0	`UNSAT` (en aplanamiento)	0	0	0	0	0
`secuencia_n3`	Borde	0	`UNSAT` (en nodo raíz)	0	1	2	1	1
`secuencia_n4`	Multim.	2	$(1, 2, 1, 0)$ y $(2, 0, 2, 0)$	5	1	37	3	3
`secuencia_n5`	Espor.	1	$(2, 1, 2, 0, 0)$	5	2	54	3	3
`secuencia_n6`	Crítico	0	`UNSATISFIABLE`	7	4	97	4	4
`secuencia_n7`	General	1	$(3, 2, 1, 1, 0, 0, 0)$	7	3	105	4	4
`secuencia_n10`	General	1	$(6, 2, 1, 0, 0, 0, 1, 0, 0, 0)$	11	5	231	7	7
`secuencia_n15`	Impar	1	$x_0 = 11, x_1 = 2, x_2 = 1, x_{11} = 1$	17	8	478	17	13
`secuencia_n20`	Escal.	1	$x_0 = 16, x_1 = 2, x_2 = 1, x_{16} = 1$	21	10	799	26	15
`secuencia_n50`	Estrés	1	$x_0 = 46, x_1 = 2, x_2 = 1, x_{46} = 1$	51	25	3 903	78	30
`secuencia_n100`	Estrés	1	$x_0 = 96, x_1 = 2, x_2 = 1, x_{96} = 1$	101	50	14 711	166	55

Tabla 4: Verificación exhaustiva (`-a`) de las 12 instancias `.dzn` sobre `secuencia.mzn` en Gecode 6.4.0 y Chuffed 0.14.0 (para $n \geq 15$, las componentes omitidas valen 0).

La Tabla 5 y la Figura 8 cuantifican el impacto individual y conjunto de R_1 y R_2. Sin restricciones redundantes (M_0), los nodos crecen cuadráticamente ($\Theta(n^2)$) y las propagaciones escalan como $\Theta(n^4)$, alcanzando 39,52 millones de propagaciones y 4,01 s en $n = 100$. Al añadir únicamente R_1 (M_1), el árbol desciende a un régimen lineal de $4n - 7$ nodos (393 en $n = 100$). Notablemente, este conteo de nodos de M_1 coincide de manera exacta con el obtenido al reemplazar los n predicados `count` por `global_cardinality_closed` sin restricciones redundantes ($11, 15, 33, 73, 193, 393$ nodos para $n = 4, 6, 10, 20, 50, 100$), debido a que una restricción de cardinalidad cerrada lleva implícita la invariante $\sum x_i = n$, pero sigue siendo ciega a los pesos posicionales $(i-1)x_i$ de R_2. Al incorporar R_2 y combinarla con R_1 en M_3, los nodos se reducen en un 99,04 % (10 557 → 101), las propagaciones en un 99,96 % (39 520 167 → 14 711) y el tiempo de búsqueda se acelera en un factor de 2 802× (4 007,51 ms → 1,43 ms) en $n = 100$.

Por su parte, la Tabla 6 contrasta las tres codificaciones de la restricción primaria de conteo manteniendo activas $R_1 + R_2$. Aunque las tres exploran exactamente el mismo árbol de búsqueda ($5, 11, 21, 51$ nodos), la suma explícita reificada multiplica por 101× el número de variables en $n = 50$ (5 050 frente a 50) y por 7,9× las propagaciones respecto a `count`. Entre `count` y `global_cardinality_closed`, la primera logra el menor tiempo de resolución práctico hasta $n = 50$ (0,50 ms frente a 0,65 ms) gracias a la ligereza aritmética de cada propagador individual, mientras que `global_cardinality_closed` minimiza el número absoluto de invocaciones del motor (509 propagaciones en $n = 50$). Finalmente, la Tabla 7 sintetiza el desempeño comparativo de las estrategias de distribución evaluadas.

n	Variante del modelo	Sol.	Nodos	Fallos	Prof.	Propagaciones	Tiempo (ms)
3	M_0: Sin redundantes	0	5	3	1	18	0,06
3	M_1: Solo R_1	0	5	3	1	21	0,06
3	M_2: Solo R_2	0	3	2	1	10	0,05
3	M_3: Ambas ($R_1 + R_2$)	0	0	1	0	2	0,04
4	M_0: Sin redundantes	2	11	4	3	50	0,08
4	M_1: Solo R_1	2	11	4	3	81	0,09
4	M_2: Solo R_2	2	5	1	2	32	0,09
4	M_3: Ambas ($R_1 + R_2$)	2	5	1	2	37	0,09
6	M_0: Sin redundantes	0	29	15	4	391	0,09
6	M_1: Solo R_1	0	15	8	3	154	0,09
6	M_2: Solo R_2	0	7	4	3	81	0,07
6	M_3: Ambas ($R_1 + R_2$)	0	7	4	3	97	0,08
10	M_0: Sin redundantes	1	117	58	6	3 739	0,35
10	M_1: Solo R_1	1	33	16	5	425	0,13
10	M_2: Solo R_2	1	11	5	5	199	0,11
10	M_3: Ambas ($R_1 + R_2$)	1	11	5	5	231	0,11
20	M_0: Sin redundantes	1	477	238	11	63 474	3,05
20	M_1: Solo R_1	1	73	36	10	1 489	0,39
20	M_2: Solo R_2	1	21	10	10	700	0,18
20	M_3: Ambas ($R_1 + R_2$)	1	21	10	10	799	0,28
50	M_0: Sin redundantes	1	2 757	1 378	26	2 482 962	149,57
50	M_1: Solo R_1	1	193	96	25	5 751	0,95
50	M_3: Ambas ($R_1 + R_2$)	1	51	25	25	3 903	0,55
100	M_0: Sin redundantes	1	10 557	5 278	51	39 520 167	4 007,51
100	M_1: Solo R_1	1	393	196	50	21 951	3,12
100	M_3: Ambas ($R_1 + R_2$)	1	101	50	50	14 711	1,43

Tabla 5: Estudio de ablación de las restricciones redundantes R_1 y R_2 en Gecode 6.4.0 bajo `first_fail` + `indomain_min` (enumeración completa `-a`).

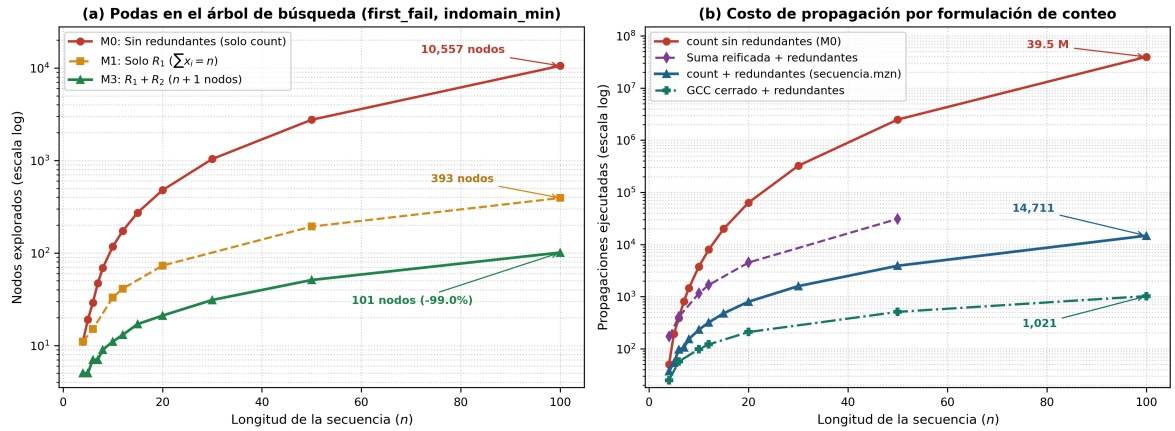


Figura 8: Curvas de escalabilidad en escala logarítmica comparando las cuatro variantes de ablación (M_0, M_1, M_2, M_3) desde $n = 3$ hasta $n = 100$: (a) nodos explorados; (b) propagaciones ejecutadas por el motor.

n	Codificación de $\mathcal{C}_{\text{base}}$ ($R_1 + R_2$ activas)	Vars.	Prop.	Nodos	Propagaciones	Tiempo (ms)
10	Suma reificada explícita	210	106	11	1 150	0,17
10	`count` por índice (modelo adoptado)	10	12	11	231	0,11
10	`global_cardinality_closed`	10	4	11	98	0,13
20	Suma reificada explícita	820	238	21	4 498	0,37
20	`count` por índice (modelo adoptado)	20	22	21	799	0,21
20	`global_cardinality_closed`	20	4	21	209	0,22
50	Suma reificada explícita	5 050	684	51	30 943	1,74
50	`count` por índice (modelo adoptado)	50	52	51	3 903	0,50
50	`global_cardinality_closed`	50	4	51	509	0,65

Tabla 6: Comparación estructural y operativa en Gecode de tres implementaciones equivalentes de la restricción básica de conteo (con $R_1 + R_2$ activas).

Estrategia de búsqueda	Sin redundantes ($n = 10$)		Con $R_1 + R_2$ (Nodos explorados)				
(Variable + Valor)	Nodos	Fallos	$n = 4$	$n = 6$	$n = 10$	$n = 50$	$n = 100$
`first_fail` + `indomain_min`	117	58	5	7	11	51	101
`first_fail` + `indomain_split`	75	37	5	7	11	51	101
`dom_w_deg` + `indomain_min`	109	54	5	7	11	51	101
`input_order` + `indomain_max`	89	44	5	7	11	51	101
Por defecto (Gecode)	107	53	5	9	11	51	101
Orden inverso + `indomain_min`	277	138	5	9	11	51	101
`input_order` + `indomain_min`	55	27	7	11	19	99	199
`smallest` + `indomain_min`	105	52	7	11	21	141	291

Tabla 7: Comparación de estrategias de distribución sin restricciones redundantes ($n = 10$) y con restricciones redundantes ($n \in \{4, 6, 10, 50, 100\}$) en Gecode 6.4.0.

3.5. Conclusiones del Ejercicio

1. **Primacía de la proyección algebraica global (R_1+R_2) sobre la propagación local:** Las restricciones básicas de conteo `count` son incapaces por sí solas de acotar globalmente las sumas ponderadas de frecuencias cuando las variables están libres, originando un crecimiento superlineal ($\Theta(n^2)$ nodos y $\Theta(n^4)$ propagaciones). La inclusión conjunta de R_1 y R_2 proyecta la estructura global del problema sobre dos hiperplanos lineales diofánticos que podan la mitad superior de los dominios a $\{0,1\}$ en la raíz, reduciendo la complejidad empírica a una ley lineal exacta de $2\lceil n/2 \rceil + 1$ nodos ($n+1$ en todo $n \geq 8$ par) y logrando una aceleración superior a tres órdenes de magnitud ($2\,802\times$) en $n = 100$.

2. **Acoplamiento entre restricciones redundantes y heurística de búsqueda:** Los experimentos demuestran que una heurística de distribución no puede seleccionarse de forma aislada respecto al modelo: `first_fail` + `indomain_min` pasa de ser subóptima en el modelo base (117 nodos en $n = 10$ debido a dominios iniciales homogéneos) a ser estrictamente óptima en todos los valores de n evaluados tras incorporar R_2, gracias a la asimetría de tamaños de dominio creada por la cota $\lfloor \frac{n-1}{i-1} \rfloor$.

3. **Superioridad de restricciones globales frente a reificación explícita y ausencia de simetrías:** Codificar el conteo mediante el predicado global `count` elimina las n^2 variables booleanas auxiliares de la formulación reificada, disminuyendo en casi un orden de magnitud las propagaciones y el tiempo de ejecución. Asimismo, se demostró rigurosamente que la naturaleza autorreferencial de x_i impide cualquier simetría de índices o valores, preservando la completitud en la enumeración de todas las secuencias mágicas.

4. Problema 4: Acertijo Lógico

4.1. Descripción del Problema y Abstracción CSP

El cuarto ejercicio plantea un problema clásico de deducción lógica combinatoria (perteneciente a la familia de problemas tipo *Zebra* o de emparejamiento multidimensional) sobre tres amigos cuyos nombres de pila son **Juan**, **Oscar** y **Darío**. Cada uno posee un único apellido dentro del conjunto $\{González, García, López\}$, una edad entera comprendida entre 24 y 26 años, y un único género musical favorito entre $\{clásica, pop, jazz\}$. El propósito consiste en determinar de forma exacta y sin ambigüedad el nombre completo, la edad y la preferencia musical de cada amigo a partir de las pistas lógicas suministradas.

El paradigma CSP($\mathcal{FD}$) se ajusta de manera natural a este tipo de acertijos deductivos porque permite expresar declarativamente cada pista como una relación matemática sobre dominios discretos finitos y delegar la inferencia al motor de consistencia local y global. En lugar de codificar árboles procedimentales *ad hoc* de descarte por casos sobre el espacio bruto de emparejamientos, la propagación combinada de desigualdades binarias y biyecciones globales (`all_different`) deduce automáticamente las exclusiones cruzadas (tales como conjuntos de Hall) y certifica simultáneamente tanto la existencia como la unicidad de la solución.

4.1.1. Pistas del Problema

El enunciado establece cinco proposiciones lógicas que restringen simultáneamente las cuatro dimensiones del problema:

1. **Pista 1:** Juan es más viejo que González, a quien le gusta la música clásica.
2. **Pista 2:** El fan de la música pop, que no es García, no tiene 24 años.
3. **Pista 3:** Oscar, quien no es López, tiene 25 años.
4. **Pista 4:** La música favorita de Darío no es el jazz.
5. **Pista 5:** Las edades de los tres amigos están entre 24 y 26 años.

Elección de la representación CSP incondicional. En problemas de emparejamiento multiatributo existen dos enfoques clásicos de modelamiento:

- *Enfoque indexado por persona (descartado por inducir condicionamiento):* definir arreglos de atributos indexados por los tres nombres de pila. Bajo dicho esquema, traducir proposiciones cruzadas entre atributos ajenos al nombre —tales como "a González le gusta la música clásica" o "el fan del pop no es García"— exige cuantificar sobre las tres personas mediante implicaciones condicionadas al valor de las variables (o indexación variable de arreglos), introduciendo disyunciones reificadas innecesarias y contraviniendo el principio de formulación directa e incondicional de restricciones.
- *Enfoque orientado a edades como dominio común de enlace (adoptado):* representar cada atributo nominal (cada nombre, cada apellido y cada género musical) mediante una variable de decisión entera cuyo valor es directamente la **edad** $e \in E = \{24, 25, 26\}$ de la persona que posee dicho atributo. Como las tres personas tienen edades distintas en E, dos atributos cualesquiera corresponden al mismo individuo si y solo si sus variables asociadas asumen exactamente el mismo valor de edad, y difieren de individuo si y solo si asumen edades distintas. De este modo, la totalidad de las pistas se traduce en igualdades y desigualdades algebraicas binarias directas, libres por completo de restricciones condicionadas.

4.1.2. Parámetros y Datos de Entrada

Al tratarse de un acertijo deductivo cerrado y autocontenido, el modelo $\mathcal{P}_{\mathrm{AL}} := \langle \mathcal{C} \ ; \ \mathcal{DE} \rangle$ **no recibe parámetros variables externos** mediante archivos de datos. Las constantes estructurales fijas del dominio del discurso quedan determinadas íntegramente por la Pista 5: el conjunto finito de edades posibles $E = \{24, 25, 26\}$ de cardinalidad $|E| = 3$.

4.1.3. Variables de Decisión y Dominios

Se definen nueve variables de decisión enteras agrupadas en tres categorías disjuntas de tres elementos cada una:

$$\begin{aligned} \mathcal{N} &= \{x_{\text{Juan}},\, x_{\text{Oscar}},\, x_{\text{Darío}}\} && \text{(edades asociadas a cada nombre de pila)}, \\ \mathcal{A} &= \{x_{\text{González}},\, x_{\text{García}},\, x_{\text{López}}\} && \text{(edades asociadas a cada apellido)}, \\ \mathcal{M} &= \{x_{\text{Clásica}},\, x_{\text{Pop}},\, x_{\text{Jazz}}\} && \text{(edades asociadas a cada género musical)}. \end{aligned}$$

Semánticamente, $x_\alpha = e$ significa que el único amigo caracterizado por el atributo $\alpha \in \mathcal{N} \cup \mathcal{A} \cup \mathcal{M}$ tiene exactamente e años de edad.

Siguiendo el criterio de excluir *a priori* de los dominios todos los valores imposibles deducibles directamente de las pistas unarias y de orden estricto (consistencia de nodo previa a la búsqueda), la declaración de dominios finitos $\mathcal{DE}$ se establece como:

$$\begin{aligned} D(x_{\text{Oscar}}) &= \{25\} && \text{(por Pista 3: Oscar tiene exactamente 25 años)}, \\ D(x_{\text{Juan}}) &= \{25, 26\} && \text{(por Pista 1: } x_{\text{Juan}} > x_{\text{González}} \geq 24 \implies x_{\text{Juan}} \neq 24), \\ D(x_{\text{González}}) &= \{24, 25\} && \text{(por Pista 1: } x_{\text{González}} < x_{\text{Juan}} \leq 26 \implies x_{\text{González}} \neq 26), \\ D(x_{\text{Pop}}) &= \{25, 26\} && \text{(por Pista 2: el fan de la música pop no tiene 24 años)}, \\ D(x_v) &= \{24, 25, 26\} && \forall v \in \{\text{Darío, García, López, Clásica, Jazz}\}. \end{aligned}$$

Esta reducción inicial encoge el espacio cartesiano bruto de $3^9 = 19\,683$ tuplas a $1 \times 2^3 \times 3^5 = 1\,944$ estados antes de evaluar una sola restricción binaria o global. Nótese que x_{Oscar} se conserva formalmente como variable de decisión con dominio unitario $\{25\}$ (y no como parámetro externo) porque las edades de los tres amigos constituyen las incógnitas endógenas del acertijo y la Pista 3 opera como una restricción unaria de fijación análoga a las casillas preasignadas de un Sudoku, preservando la homogeneidad tipada de la biyección $\mathcal{N}$.

4.1.4. Restricciones del Modelo

El conjunto de restricciones $\mathcal{C}$ se compone de dos familias puramente algebraicas e incondicionales:

1. Restricciones estructurales de biyección por categoría. Cada uno de los tres amigos posee un nombre distinto, un apellido distinto, un género musical distinto y una edad distinta en $E = \{24, 25, 26\}$:

$$B_{\mathcal{N}} : \quad \text{all_different}(x_{\text{Juan}},\, x_{\text{Oscar}},\, x_{\text{Darío}}), \tag{5}$$

$$B_{\mathcal{A}} : \quad \text{all_different}(x_{\text{González}},\, x_{\text{García}},\, x_{\text{López}}), \tag{6}$$

$$B_{\mathcal{M}} : \quad \text{all_different}(x_{\text{Clásica}},\, x_{\text{Pop}},\, x_{\text{Jazz}}). \tag{7}$$

Justificación formal de la unicidad de edades: El supuesto estructural de que los tres amigos tienen edades mutuamente distintas entre $\{24, 25, 26\}$ no solo es imprescindible para que la edad actúe como clave biyectiva entre atributos, sino que es matemáticamente necesario para que el acertijo posea solución única. En efecto, si se permitiesen edades repetidas (modelando por

persona sin exigir edades distintas), la asignación donde Juan tiene 26 años y tanto Oscar como Darío tienen 25 años permitiría que Darío sea González (25 años, música clásica), porque la Pista 2 ("el fan del pop no tiene 24") dejaría de excluir el emparejamiento cuando nadie tiene 24 años. La verificación experimental confirma que omitir la unicidad de edades eleva el conjunto de soluciones de 1 a 6 asignaciones distintas.

2. Restricciones relacionales derivadas de las pistas. Cada proposición del enunciado se proyecta directamente en relaciones binarias de orden, igualdad o desigualdad:

$$
\begin{aligned}
C_{1a}: &\quad x_{\text{Juan}} > x_{\text{González}} && \text{(Pista 1: Juan es mayor que González)}, && (8)\\
C_{1b}: &\quad x_{\text{González}} = x_{\text{Clásica}} && \text{(Pista 1: a González le gusta la música clásica)}, && (9)\\
C_{2a}: &\quad x_{\text{Pop}} \neq x_{\text{García}} && \text{(Pista 2: el fan del pop no es García)}, && (10)\\
C_{2b}: &\quad x_{\text{Pop}} \neq 24 && \text{(Pista 2: absorbida en } D(x_{\text{Pop}}) = \{25, 26\}), && (11)\\
C_{3a}: &\quad x_{\text{Oscar}} \neq x_{\text{López}} && \text{(Pista 3: Oscar no se apellida López)}, && (12)\\
C_{3b}: &\quad x_{\text{Oscar}} = 25 && \text{(Pista 3: absorbida en } D(x_{\text{Oscar}}) = \{25\}), && (13)\\
C_{4}: &\quad x_{\text{Darío}} \neq x_{\text{Jazz}} && \text{(Pista 4: la música favorita de Darío no es el jazz)}. && (14)
\end{aligned}
$$

4.2. Detalles de Implementación (`acertijo.mzn`)

En el archivo `problemas/2.4/acertijo.mzn` se declaran las nueve variables enteras con sus dominios finitos podados, se agrupan en tres arreglos unidimensionales por categoría para aplicar el predicado global `alldifferent` y reconstruir la salida tabular mediante comprensión declarativa por edad, y se imponen las cinco restricciones binarias activas ($C_{1a}, C_{1b}, C_{2a}, C_{3a}, C_4$).

Para evaluar rigurosamente la calidad de esta implementación, se contrastaron tres variantes de codificación:

1. **Modelo incondicional con `alldifferent` global (`acertijo.mzn`):** produce únicamente 7 variables enteras en FlatZinc (tras unificar la constante $x_{\text{Oscar}} = 25$ y la igualdad $x_{\text{González}} = x_{\text{Clásica}}$), 0 variables booleanas reificadas y resuelve el problema íntegramente en la fase de propagación inicial con 9 propagaciones y 0 propagadores residuales.

2. **Descomposición binaria por pares de desigualdades ($\neq$):** reemplaza cada predicado global `alldifferent` de tamaño 3 por $\binom{3}{2} = 3$ desigualdades binarias (9 en total). Aunque evita variables booleanas, pierde la capacidad de detectar conjuntos de Hall de cardinalidad 2, dejando 7 propagadores activos tras la raíz y obligando al motor a ramificar.

3. **Modelo alternativo condicionado por persona (evaluado como línea base de contraste):** al indexar apellido y música por persona usando implicaciones reificadas, el compilador genera 28 variables en el motor (18 booleanas auxiliares de semirreificación), 26 propagadores y 70 propagaciones (7,8× más trabajo operativo que `acertijo.mzn`), confirmando cuantitativamente la superioridad del modelo orientado a edades.

4.2.1. Restricciones Redundantes

Dado que cada categoría $\mathcal{K} \in \{\mathcal{N}, \mathcal{A}, \mathcal{M}\}$ constituye una permutación biyectiva del conjunto cerrado $E = \{24, 25, 26\}$, toda solución válida satisface necesariamente las invariantes lineales de suma constante y la transitividad de orden entre C_{1a} y C_{1b}:

$$
R_{\Sigma,\mathcal{K}}: \sum_{v \in \mathcal{K}} x_v = 24 + 25 + 26 = 75 \quad (\forall \mathcal{K} \in \{\mathcal{N}, \mathcal{A}, \mathcal{M}\}), \qquad R_{\text{ord}}: \; x_{\text{Juan}} > x_{\text{Clásica}}.
$$

Sin embargo, el análisis teórico y experimental demuestra que **no es conveniente ni necesario incluir dichas restricciones redundantes en `acertijo.mzn`**:

- Bajo el predicado global `alldifferent`, la consistencia de hiper-arco (dominio) subsume por completo la poda de cotas de $\sum_{v \in \mathcal{K}} x_v = 75$ y resuelve el problema en el nodo raíz con 1 nodo y 0 fallos, por lo que cualquier restricción adicional solo añadiría sobrecarga de inicialización.

- Incluso bajo la descomposición binaria ($\neq$), donde en la raíz se tiene $D(x_{\text{Pop}}) = D(x_{\text{Jazz}}) = \{25, 26\}$ y $D(x_{\text{Clásica}}) = \{24, 25\}$, añadir la suma redundante $x_{\text{Clásica}} + x_{\text{Pop}} + x_{\text{Jazz}} = 75$ tampoco logra podar el valor 25 de $D(x_{\text{Clásica}})$, puesto que la consistencia de cotas sobre una ecuación lineal evalúa independientemente los extremos de cada variable ($75 - \max D(x_{\text{Pop}}) - \max D(x_{\text{Jazz}}) = 75 - 26 - 26 = 23 \leq x_{\text{Clásica}} \leq 75 - 25 - 25 = 25$), ignorando que x_{Pop} y x_{Jazz} no pueden valer 25 simultáneamente. Experimentalmente, agregar las sumas redundantes al modelo binario mantiene inalterados los 3 nodos y 1 fallo, pero incrementa las propagaciones de 12 a 22 (+83,3 %).

4.2.2. Rompimiento de Simetrías

El modelo $\mathcal{P}_{\text{AL}}$ **carece por completo de simetrías de variables y de valores**:

1. **Distinguibilidad semántica de variables:** Cada variable representa una entidad nominal única e irrepetible del mundo real (Juan, Oscar, Darío, González, García, López, clásica, pop, jazz) sometida a restricciones asimétricas individuales ($x_{\text{Oscar}} = 25$, $x_{\text{Juan}} > x_{\text{González}}$, $x_{\text{Pop}} \neq 24$, $x_{\text{Darío}} \neq x_{\text{Jazz}}$). Ninguna permutación no trivial π de variables deja invariante la estructura sintáctica de $\mathcal{C}$ ni los dominios $\mathcal{DE}$.

2. **Asimetría de valores y solución única:** Los tres valores del dominio $\{24, 25, 26\}$ poseen orden total aritmético explotado por la desigualdad estricta $x_{\text{Juan}} > x_{\text{González}}$ y por las constantes fijadas (25 y $\neq 24$). Además, como $|\text{Sol}(\mathcal{P}_{\text{AL}})| = 1$, la órbita del conjunto de soluciones bajo cualquier grupo de transformaciones es trivial de cardinalidad uno, haciendo inaplicable cualquier restricción de rompimiento de simetrías.

4.3. Estrategias de Búsqueda y Distribución

En `acertijo.mzn` se especifica una estrategia de distribución completa sobre el vector concatenado de las nueve variables de decisión empleando `first_fail` (selección de la variable no fijada con menor dominio actual) combinada con `indomain_min` (asignación ascendente del mínimo valor disponible). Adicionalmente, se compararon siete estrategias combinando `input_order`, `first_fail`, `dom_w_deg` y la búsqueda por defecto del solver con `indomain_min`, `indomain_max` e `indomain_split`, tanto en modo de primera solución (`-n 1`) como de enumeración exhaustiva (`-a`).

La Figura 9 expone con precisión el mecanismo deductivo subyacente y la diferencia exacta entre consistencia global y consistencia de arco binaria:

1. **Resolución sin búsqueda bajo `alldifferent` global (Figura 9a):**

   - *Paso 1 (Nombres):* Con $D(x_{\text{Oscar}}) = \{25\}$ y $D(x_{\text{Juan}}) = \{25, 26\}$, la desigualdad con Oscar fija $x_{\text{Juan}} = 26$; enseguida, $B_{\mathcal{N}}$ fuerza por exclusión $x_{\text{Darío}} = 24$.
   - *Paso 2 (Conjunto de Hall en Música):* Al propagarse $x_{\text{Darío}} = 24$ hacia C_4 ($x_{\text{Darío}} \neq x_{\text{Jazz}}$), el valor 24 se elimina de $D(x_{\text{Jazz}})$, quedando $D(x_{\text{Jazz}}) = \{25, 26\}$. Unido a $D(x_{\text{Pop}}) = \{25, 26\}$, el subconjunto de dos variables $\{x_{\text{Pop}}, x_{\text{Jazz}}\}$ posee como unión de dominios exactamente el conjunto $\{25, 26\}$ de cardinalidad 2 (**conjunto de Hall de tamaño** 2).
   - *Paso 3 (Poda de hiper-arco y cascada final):* El propagador global de $B_{\mathcal{M}}$ detecta el conjunto de Hall y poda $\{25, 26\}$ de $D(x_{\text{Clásica}})$, fijando $x_{\text{Clásica}} = 24$. Por C_{1b} se instancia $x_{\text{González}} = 24$; como C_{3a} ($x_{\text{Oscar}} \neq x_{\text{López}}$) ya había podado 25 de

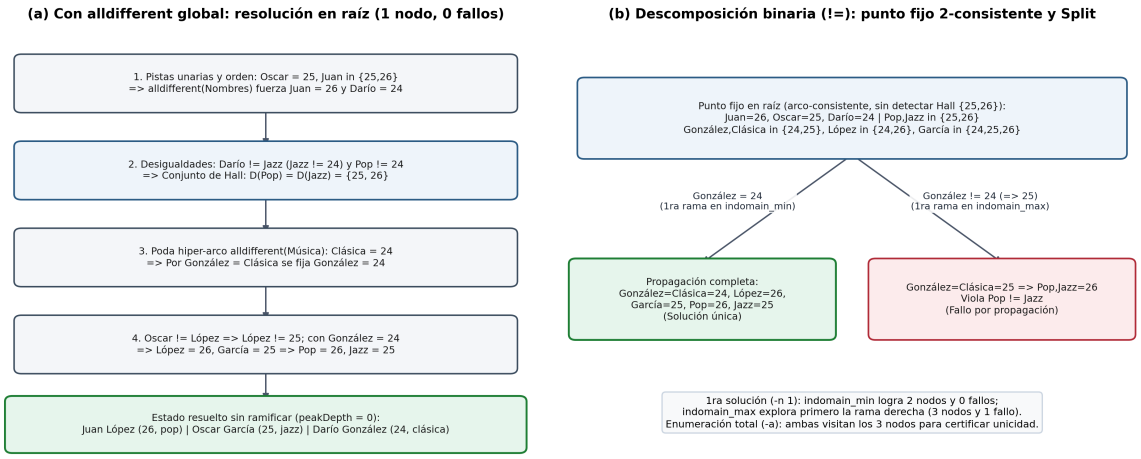


Figura 9: Contraste operativo en el Acertijo Lógico: (a) cascada deductiva determinista en el nodo raíz con `alldifferent` global (1 nodo, 0 fallos, profundidad 0); (b) árbol binario con descomposición por pares ($\neq$), donde el punto fijo 2-consistente exige ramificar sobre $x_{\text{González}} \in \{24, 25\}$.

$D(x_{\text{López}}) = \{24, 26\}$, la biyección $B_{\mathcal{A}}$ fija $x_{\text{López}} = 26$ y $x_{\text{García}} = 25$. Finalmente, C_{2a} ($x_{\text{Pop}} \neq x_{\text{García}} = 25$) fija $x_{\text{Pop}} = 26$ y $B_{\mathcal{M}}$ concluye $x_{\text{Jazz}} = 25$. Como todas las variables quedan univaluadas en la raíz (peakDepth $= 0$), las siete estrategias de distribución empatan exactamente en 1 **nodo explorado y** 0 **fallos** (incluso desactivando la optimización del compilador con `-O0`).

2. **Aparición de búsqueda bajo descomposición binaria ($\neq$) (Figura 9b):** Cuando $B_{\mathcal{M}}$ se descompone en las tres desigualdades binarias $x_{\text{Clásica}} \neq x_{\text{Pop}}$, $x_{\text{Clásica}} \neq x_{\text{Jazz}}$ y $x_{\text{Pop}} \neq x_{\text{Jazz}}$, cada restricción binaria es individualmente arco-consistente sobre $D(x_{\text{Pop}}) = D(x_{\text{Jazz}}) = \{25, 26\}$ y $D(x_{\text{Clásica}}) = \{24, 25\}$ (pues para cada valor en una variable existe soporte distinto en la otra). El motor alcanza así un punto fijo 2-consistente no resuelto con cinco variables residuales y debe ejecutar un *Split* sobre $x_{\text{González}} \in \{24, 25\}$:

   - Al buscar **la primera solución** (`-n 1`), las estrategias con selección mínima o bisección (tales como `first_fail` o `input_order` combinadas con `indomain_min`, `indomain_split` y `dom_w_deg`) eligen primero la rama izquierda $x_{\text{González}} = 24$, hallando la solución de inmediato en 2 **nodos,** 0 **fallos y** 7 **propagaciones**; en cambio, las heurísticas de valor máximo (`indomain_max`) exploran primero la rama derecha $x_{\text{González}} = 25$, la cual obliga $x_{\text{Clásica}} = 25 \Rightarrow x_{\text{Pop}} = x_{\text{Jazz}} = 26$ (fallo por violar $x_{\text{Pop}} \neq x_{\text{Jazz}}$), retrocediendo hacia $x_{\text{González}} = 24$ con 3 **nodos,** 1 **fallo y** 12 **propagaciones**.
   - Al exigir **enumeración exhaustiva** (`-a`) para certificar unicidad, toda estrategia completa debe inspeccionar ambas ramas del punto de decisión, convergiendo en 3 nodos, 1 fallo y 12 propagaciones (16 propagaciones si no se acotan los dominios iniciales *a priori*).

4.4. Pruebas y Resultados

La verificación experimental de `problemas/2.4/acertijo.mzn` (automatizada de forma reproducible en la suite `pruebas/2.4/test_acertijo.py`) se llevó a cabo en tres niveles complementarios: (i) validación cruzada de la solución única entre **Gecode 6.4.0** y **Chuffed 0.14.0**, obteniendo exactamente la misma asignación biyectiva y verificando programáticamente las cinco pistas sobre la salida (Tabla 8); (ii) evaluación comparativa de codificaciones y estrategias de

distribución (Tabla 9); y (iii) una batería de pruebas de **minimalidad e independencia de pistas** mediante ablación controlada y perturbación contradictoria (Tabla 10).

Nombre de pila	Apellido	Edad (años)	Género musical favorito
Juan	López	26	Pop
Oscar	García	25	Jazz
Darío	González	24	Clásica

Tabla 8: Solución única identificada y certificada para el Acertijo Lógico.

Es inmediato comprobar manualmente que la Tabla 8 satisface las cinco pistas: (1) Juan (26) es mayor que Darío González (24), quien prefiere música clásica; (2) el fan del pop es Juan López (no García) y tiene $26 \neq 24$ años; (3) Oscar se apellida García (no López) y tiene 25 años; (4) la música favorita de Darío es clásica (no jazz); y (5) las edades son $\{24, 25, 26\}$.

Codificación del modelo (Solver)	Estrategia de búsqueda	Vars. motor	Prop. act.	1ra (`-n 1`) Nod.	1ra (`-n 1`) Fall.	Todas (`-a`) Nod.	Todas (`-a`) Fall.	Todas (`-a`) Propag.
`alldifferent` (Gecode)	Todas (7 eval.)	9	0	1	0	1	0	9
`alldifferent` (Chuffed)	`first_fail` + `min`	55	10	1	0	1	1	11
Binaria ($\neq$), dom. podado	`first_fail` + `min`	8	7	2	0	3	1	12
Binaria ($\neq$), dom. podado	`input_order` + `min`	8	7	2	0	3	1	12
Binaria ($\neq$), dom. podado	`first_fail` + `split`	8	7	2	0	3	1	12
Binaria ($\neq$), dom. podado	`dom_w_deg` + `min`	8	7	2	0	3	1	12
Binaria ($\neq$), dom. podado	`first_fail` + `max`	8	7	3	1	3	1	12
Binaria ($\neq$), dom. podado	`input_order` + `max`	8	7	3	1	3	1	12
Binaria ($\neq$), dom. $\{24, 25, 26\}$	`first_fail` + `min`	8	7	2	0	3	1	16
Binaria ($\neq$) + $\sum = 75$	`first_fail` + `min`	8	9	2	0	3	1	22
Condicionado por persona	`first_fail` + `min`	28	26	2	0	3	1	70

Tabla 9: Desempeño comparativo de codificaciones y estrategias de distribución para el Acertijo Lógico (tiempos de resolución entre 0,07 ms y 1,03 ms). En Chuffed `-a`, el fallo reportado corresponde a la refutación final de completitud tras hallar la única solución.

Finalmente, la Tabla 10 demuestra que el sistema de pistas del enunciado es **minimal e irredundante**: al suprimir individualmente cualquiera de las cuatro desigualdades ($x_{\text{Pop}} \neq 24$, $x_{\text{Darío}} \neq x_{\text{Jazz}}$, $x_{\text{Oscar}} \neq x_{\text{López}}$ o $x_{\text{Pop}} \neq x_{\text{García}}$), el problema deja de resolverse en la raíz y pasa a admitir exactamente 2 **soluciones válidas** (3 nodos, 0 fallos, profundidad 1). Por ejemplo, al relajar $x_{\text{Pop}} \neq 24$, emerge la solución espuria adicional donde Oscar es González (25, clásica), Juan es García (26, jazz) y Darío es López (24, pop). Por el contrario, al inyectar la restricción contradictoria $x_{\text{Pop}} = x_{\text{García}}$, el solver certifica insatisfacibilidad (`UNSATISFIABLE`) en 0 soluciones.

4.5. Conclusiones del Ejercicio

1. **Eliminación total de reificación mediante el dominio de enlace por edades:** Escoger la edad $E = \{24, 25, 26\}$ como valor de las variables asociadas a cada nombre, apellido y género musical transforma todas las relaciones cruzadas del acertijo en restricciones algebraicas directas sin condicionamiento, reduciendo en un 87,1 % las propagaciones (9 frente a 70) y eliminando las 18 variables booleanas auxiliares requeridas por un modelo condicionado indexado por persona.

2. **Poder deductivo de la consistencia de hiper-arco frente a la descomposición**

Caso de prueba experimental	Efecto semántico verificado	Sol.	Nodos	Fallos	Propag.
Modelo completo (`acertijo.mzn`)	Unicidad exacta	1	1	0	9
Sin Pista 2b ($x_{\text{Pop}} \neq 24$)	Relaja dominio de Pop	2	3	0	13
Sin Pista 4 ($x_{\text{Darío}} \neq x_{\text{Jazz}}$)	Ambigüedad Clásica/Jazz	2	3	0	12
Sin Pista 3a ($x_{\text{Oscar}} \neq x_{\text{López}}$)	Ambigüedad García/López	2	3	0	16
Sin Pista 2a ($x_{\text{Pop}} \neq x_{\text{García}}$)	Ambigüedad Pop/Jazz	2	3	0	11
Sin biyección de edades distintas	Colapso de clave biyectiva	6	11	0	29
Contradictoria ($x_{\text{Pop}} = x_{\text{García}}$)	Instancia insatisfacible	0	0	1	4

Tabla 10: Batería de pruebas de validación, minimalidad de pistas e insatisfacibilidad en Gecode 6.4.0 (`-a`).

binaria: La restricción global `alldifferent` detecta el conjunto de Hall $\{x_{\text{Pop}}, x_{\text{Jazz}}\} \subseteq \{25, 26\}$ y resuelve completamente el acertijo en el nodo raíz (1 nodo, 0 fallos, profundidad 0), volviendo invariante el desempeño frente a la estrategia de distribución. En cambio, la descomposición en desigualdades binarias ($\neq$) queda atrapada en un punto fijo 2-consistente que exige ramificar sobre $x_{\text{González}}$, haciendo que la elección del valor inicial (`indomain_min` frente a `indomain_max`) determine la presencia o ausencia de fallos al buscar la primera solución.

3. **Minimalidad de las pistas e irrelevancia operativa de restricciones redundantes y simetrías:** El estudio de ablación certificó que cada una de las pistas del enunciado (incluida la distinción biyectiva de edades) es estrictamente indispensable para garantizar unicidad, y que añadir invariantes redundantes de suma ($\sum = 75$) no aporta poda adicional ni siquiera al modelo binario por limitaciones intrínsecas de la consistencia de cotas lineal.

5. Problema 5: Ubicación de Personas en una Reunión

5.1. Descripción del Problema y Abstracción CSP

Se tiene un grupo de N personas que quieren organizarse para una foto. Las personas puede establecer una serie de preferencias sobre su posición con respecto a otras personas. En particular se establecen tres clases de preferencias:

1. *Next(A,B):* Establece que la persona A debe estar junto a la persona B.
2. *Separate(A,B):* Establece que la persona A y B deben estar separados en la foto.
3. *Distance(A, B, M):* Establece que la persona A y B deben estar separadas por máximo M personas.

5.1.1. Parámetros y Estructura de Entrada

Para los datos de entrada se emplean un total de 4 arreglos, el primero define la cantidad, nombre y posición inicial de las personas a organizar para la foto, y los demás 3 definen las distintas restricciones a tener en cuenta:

- **Personas:** Se define un arreglo de dimensión desconocida n de cadenas de caracteres *strings* para recibir los nombres de las personas a organizar en foto.

  $$P = (p_i)_{1 \leq i \leq n} \ \ con \ \ p_i \in \Sigma^*$$

  Para $\Sigma = \{a, b, c \cdots\}$ y n la longitud del arreglo, la cual representa la cantidad de personas a ordenar.

  Codificad en Minizinc por medio de la expresión:

  ```
  array[int] of string: personas;
  ```

  Observe que el arreglo se define con dimensión `int` dado que inicialmente no se conoce la cantidad exacta de personas a organizar.

- **Next:** Dado que Minizinc no permite la definición de arreglos anidados, es decir un arreglo de arreglos, se emplea un arreglo 2D de enteros para almacenar la totalidad de las restricciones Next de entrada. En el arreglo cada fila representa una preferencia, por lo que el arreglo tiene dimensión $k \times 2$ dado que cada preferencia next tiene únicamente dos valores, que corresponden a las posiciones en el arreglo *personas* de la dupla que prefiere estar juntos en la foto. Adicionalmente, dado que k corresponde al número de preferencias *Next* contempladas en la instancia del problema asumiendo que no se contemplan ninguna preferencia redundante o duplicada k se encuentra acotada por $k \leq \binom{n}{2} = \frac{n(n-1)}{2}$ para un total de n personas. El arreglo se define entonces como:

  $$X = (x_{i,j})_{1 \leq i \leq 2, \ 0 \leq j \leq \frac{n(n-1)}{2}} \ \ con \ \ x_{i,j} \in [1, n] \wedge x_{i,j} \neq x_{i+1,j}$$

  Observe que dado que una persona no puede preferir estar junto a sí misma en la foto, dentro de una misma fila los elementos del arreglo deben ser distintos, definiendo una preferencia entre dos personas distintas. El arreglo se incluyen en el modelo en Minizinc por medio de:

  ```
  array[int, 1..2] of 1..length(personas): next;
  ```

De nuevo con una dimensión desconocida dado que su longitud depende del número de preferencias incluidas en la instancia del problema, y delimitando el valor de los elementos del arreglo al intervalo $[1, n]$

- **Separate:** Definido de forma idéntica **Next**, se emplea un arreglo bidimensional de enteros para representar las preferencias $Separate(A, B)$ incluidas en el problema. Tiene de igual forma dimensión $l \times 2$ con $0 \leq l \leq \frac{n(n-1)}{2}$:

$$R = (r_{i,j})_{1 \leq i \leq 2,\ 0 \leq j \leq \frac{n(n-1)}{2}} \ \ con\ \ r_{i,j} \in [1, n] \land r_{i,j} \neq r_{i+1,j}$$

Teniendo en cuenta de nuevo que una persona no puede sentarse a parte de sí misma.

El arreglo se implementa en Minizinc de forma equivalente:

```
array[int, 1..2] of 1..length(personas): separate;
```

- **Distance:** Por último, para recibir las preferencias de tipo $distance(A, B, m)$, se empleó igualmente un arreglo de dos dimensiones, pero esta vez de tamaño $h \times 3$ pues una preferencia de distancia tiene tres párametros, la posición de las dos personas en el arreglo de personas y el valor m representando la cantidad máxima de personas que pueden separar a A y B.

Observe que una vez más, al establecer una preferencia para la posición de dos personas el número de preferencias de distancia posibles se encuentra acotado $0 \leq h \leq \frac{n(n-1)}{2}$. Aún cuando dos preferencias del tipo (A, B, m_1) y (A, B, m_2) para $m_1 \neq m_2$ se consideran preferencias diferentes, la consideración de ambas al tiempo resultaría en un problema sobre-restringido sin solución posible, y un caso dónde $m_1 = m_2$ simplemente establecería la misma preferencia dos veces. Por lo que igual es razonable emplear la misma cota para el número máximo de preferencias de distancia entre n personas. De forma que definimos el arreglo como:

$$D = (d_{i,j})_{1 \leq i \leq 3,\ 0 \leq j \leq \frac{n(n-1)}{2}} \ \ con\ \ d_{i,j} \in [1, n] \land i = 1 \rightarrow d_{i,j} \neq d_{i+1,j}$$

Observe que podemos definir la totalidad del dominio del arreglo como el intervalo $[1, n]$ dado que a lo sumo dos personas pueden situarse a lo sumo a una distancia de $n - 1$ personas entre sí, por lo que una preferencia con $m > n$ se cumpliría siempre y no tendría sentido explicitarla.

Por último, igualmente es útil también impedir una preferencia dónde $A = B$ dado que de todas formas una persona siempre está sentada a 0 personas de distancia de sí misma y todas las preferencias de este tipo se cumplirían siempre. Por lo que imponemos $i = 1 \rightarrow d_{i,j} \neq d_{i+1,j}$ para impedir que los dos primeros valores de una fila coincidan. Sobre la tercera columna del arreglo, que representa la cantidad máxima de personas que pueden separar a A de B no se impone ninguna restricción a excepción del dominio $m \in [1, n]$ pues la preferencia puede explicitar cualquier valor de distancia.

Este arreglo se implementó dentro de Minizinc como:

```
array[int, 1..3] of 1..length(personas): distance;
```

5.1.2. Variables de Decisión y Dominios

Como variable de decisión se determinó emplear un único arreglo unidimensional de longitud igual al arreglo de personas conteniendo enteros del 1 a n, representando la posición en la fila de la foto de las personas en el arreglo original de personas. Por ejemplo, para un arreglo de personas ["$Carlos$","$Maria$","$Juan$"] un arreglo de solución $[2, 3, 1]$ representa que en el orden de solución final para la foto, Carlos va de segundo, Maria de tercera y Juan de primero. Con lo que el orden para la foto corresponde a Juan-Maria-Carlos.

Dado que cada persona sólo puede ocupar un único lugar los valores del arreglo son las posiciones del arreglo original de personas, es decir que están en el intervalo $[1, n]$. Por lo que lo definimos de forma similar al arreglo de personas:

$$S = (s_i)_{1 \leq i \leq n} \ con \ \ s_i \in [1, n]$$

Para la implementación en Minizinc, el arreglo se define de igual longitud que el arreglo de Personas por medio del operador `index_set()` para extraer las posiciones del arreglo de personas como un arreglo:

```
array[index_set(personas)] of var 1..length(personas): posiciones;
```

5.1.3. Restricciones del Modelo

Para la definición del modelo del problema se emplearon un total de 4 restricciones:

1. **Posiciones únicas:** Dado que cada valor en el arreglo de solución representa la posición que ocupa una persona en la fila de la foto, dado que dos personas no pueden ocupar la misma posición, todos los valores deben ser distintos:

   $$\forall i, j \in \{1, \cdots, n\} : \ \ i \neq j \rightarrow s_i \neq s_j$$

   De forma que cada par de valores en el arreglo de solución deben ser distintos. Observe que dado que el arreglo de solución S es igualmente de tamaño n, y todos sus valores deben ser distintos con dominio de 1 a n, se garantiza que todas las personas aparezcan en la foto y ocupen un lugar válido, pues se deben tener n valores distintos en un arreglo de longitud n de dominio $[1, n]$.

   En Minizinc la restricción se implementa simplemente por medio de la restricción global `all_different`:

   ```
   %Todas las posiciones diferentes
   constraint all_different(posiciones);
   ```

2. **Next to:** Para imponer la restricciones de las preferencias de tipo *junto a* se debe imponer que por cada fila del arreglo *next* la distancia entre las personas sea igual a 1. Para ello, se puede emplear el valor absoluto de la resta entre las posiciones de las personas correspondientes en el arreglo de solución. Dado que los valores de este arreglo representa la posición de las personas, su diferencia permite conocer la distancia entre ellas. Y una distancia igual a 1 corresponde a dos personas una junta a otra:

   $$\forall i \in \{1, \cdots, k\} : \ \ |s_{x_{i,1}} - s_{x_{i,2}}| = 1 \ \ con \ \ X \in S^{k \times 2}$$

   De esta forma, cada fila del arreglo de preferencias **Next** X define una restricción estableciendo que la distancia entre las personas A y B en el arreglo de solución debe ser

exactamente 1. Efectivamente imponiendo que estén una junto a otra.

Para implementar en Minizinc se emplea el operador `index_set_1of2` que retorna el arreglo de posiciones de la primera dimensión de un arreglo 2D, permitiendo iterar sobre las filas del arreglo de preferencias:

```
%Next to:
constraint forall(i in index_set_1of2(next)) (
  abs(posiciones[next[i,1]] - posiciones[next[i,2]]) = 1
);
```

3. **Separate:** De forma similar, para implementar las preferencias de separación, se debe establecer que en el arreglo de solución la diferencia entre los valores en la posición que representa a las personas Ay B al interior de la preferencia debe ser en este caso, explícitamente distinto de 1. Impiendo que las personas se encuentran una junto a la otra:

$$\forall i \in \{1, \cdots, l\} : \ |s_{r_{i,1}} - s_{r_{i,2}}| \neq 1 \ \ con \ \ R \in S^{l \times 2}$$

En Minizinc, la restricción se implementa de forma equivalente:

```
%Separate:
constraint forall(i in index_set_1of2(separate)) (
  abs(posiciones[separate[i,1]] - posiciones[separate[i,2]]) != 1
);
```

4. **Distance:** Por último, para implementar las preferencias de distancia, se emplea la misma estructura de las restricciones anteriores, pero en este caso la distancia entre las personas en la foto debe ser máximo $m + 1$. Puesto que para dos personas con $m + 1$ distancia entre ellos en la fila, hay m entre ellos. Por ejemplo, si dos personas se encuentran uno junto a otro, están a una distancia de 1 pero hay cero personas entre ellos. Igualmente, si dos personas están a una distancia de 2, están separados por 1 persona. Por ello, una separación máxima de m personas corresponde con una distancia máxima de $m + 1$.

Este valor m lo obtenemos a partir del valor de la tercera columna en el arreglo de preferencias:

$$\forall i \in \{1, \cdots, h\} : \ |s_{d_{i,1}} - s_{d_{i,2}}| = d_{i,3} + 1 \ \ con \ \ D \in S^{h \times 3}$$

Y de nuevo, para la implementación en Minizinc podemos emplear el operador `index_set_1of2` para recorrer las filas del arreglo de preferencias:

```
%Distances:
constraint forall(i in index_set_1of2(distance)) (
  abs(posiciones[distance[i,1]] - posiciones[distance[i,2]]) <= distance[i,3] + 1
);
```

5.2. Detalles de Implementación (`reunion.mzn`)

A continuación se detallan algunas de las características adicional tenidas en cuentas durante la codificación del modelo de restricciones para el problema de la Reunión:

5.2.1. Diferentes Implementaciones de Restricciones

Una implementación alternativa de las restricciones del modelo se puede lograr al cambiar el significado de la variable de decisión. Consideremos que la fila se puede visualizar como una lista enlazada en dónde cada persona conoce quien es su vecino de la derecha, así como variable de decisión se define el arreglo *sig* en dónde la posición $sig[i]$ corresponde a la persona que está a la derecha de la persona i:

```
% sig[A] es la persona que está inmediatamente a la derecha de A (0 si es el final de la
array[0..N] of var 0..N: sig;
```

Observe que debemos emplear un nodo fantasma, una persona 0 para poder considerar el arreglo como un anillo cerrado, de esta forma definimos que el valor $sig[i] = 0$, indica que la persona i se encuentra al final de la fila dado que no tiene ningún vecino y apunta al nodo fantasma y de igual forma el valor se $sig[0]$ corresponde a la primera persona en la fila.

Para garantizar que el arreglo forme efectivamente un camino cerrado con la totalidad de nodos, dónde todos son sucesor de algún, empleamos el *constraint circuit* de entre las restricciones a grafos [**?**]:

```
include "circuit.mzn";
%Circuito
constraint circuit(sig);
```

De esta forma, la restricciones de posiciones diferentes se puede imponer obligando también a que todo el arreglo de vecinos sea también distinto, puesto que si todas las personas tienen vecinos distintos, pues todos ocupan un lugar distinto en la fila. Puesto que si ambas personas comparten el mismo vecino, deberían estar ocupando el mismo lugar en la fila, lo que no es deseable.

```
   %Todos los vecinos diferentes
constraint all_different(sig);
```

También se garantiza que todas las personas ocupen un lugar en la fila por medio de la longitud del arreglo, dado que cada posición representa una persona en la fila.

Por medio de esta nueva variable de decisión, la implementación de las restricciones de tipo *Next to* y *Separate* se tornan mucho más inmediatas, pues corresponden simplemente a imponer restricciones sobre el valor en el arreglo *sig* de las posiciones de las personas A y B, sin necesidad de operaciones adicionales de resta o valor absoluto:

```
% Next to (A está inmediatamente antes que B OR B inmediatamente antes que A)
constraint forall(i in index_set_1of2(next)) (
  sig[next[i,1]] == next[i,2] \/ sig[next[i,2]] == next[i,1]
);

% Separate (A no puede ser sucesor directo de B ni viceversa)
constraint forall(i in index_set_1of2(separate)) (
  sig[separate[i,1]] != separate[i,2] /\ sig[separate[i,2]] != separate[i,1]
);
```

Con ello, las preferencias *Next to* se reduce a obligar a que $sig[A] = B$ o $sig[B] = A$ garantizando que sean vecinos uno de otro. Y de forma equivalente, una preferencia *Separate* se impone simplemente estableciendo que $sig[A]! = B$ y $sig[B]! = A$ garantizando que no sean vecinos. Se

establece una restricción por cada fila de los arreglos `next` y `separate` igualmente por medio del arreglo de posiciones arrojado por medio de `index_set_1of2()`.

Para la restricción de distancias podemos definir una función auxiliar que indique si un nodo es alcanzable desde otro en un número máximo de pasos, de esta forma, podemos validar si entre la persona A y B hay un máximo de $m+1$ pasos o m personas:

```
    % Función recursiva/desplegada para verificar alcance en K pasos
predicate alcanzable(array[int] of var int: s, var int: actual,
    int: destino, int: pasos_restantes) =
  if pasos_restantes <= 0 \/ actual == 0 then
    false
  else
    s[actual] == destino \/ (s[actual] != 0 /\
        alcanzable(s, s[actual], destino, pasos_restantes - 1))
  endif;
```

Observe que para evitar que las soluciones empleen el nodo fantasma como un camino válido se define explícitamente que llegar o pasar por el nodo fantasma resulta en un camino falso, por lo que la restricción se cumple únicamente si se puede llegar de la persona A a B en máximo $m+1$ pasos sin pasar por el nodo fantasma, dado que según la formulación original del problema, las personas forman una fila y no un anillo cerrado.

Lo único que falta es verificar que alguno de los dos caminos $A \to B$ o $B \to A$ sea posible para cada de una de las preferencias de distancia:

```
    constraint forall(i in index_set_1of2(distance)) (
    alcanzable(sig, distance[i,1], distance[i,2], distance[i,3] + 1) \/
    alcanzable(sig, distance[i,2], distance[i,1], distance[i,3] + 1)
  );
```

Finalmente, para recuperar el resultado, basta con empezar en $sig[0]$ y recorrer el arreglo juntando el orden de las personas hasta llegar al nodo con $sig[n] = 0$:

```
    % Función recursiva: avanza por los sucesores hasta encontrar el nodo 0 (fin)
function array[int] of string: obtener_fila(array[int] of int: s, int: actual) =
  if s[actual] == 0 then
    []
  else
    [personas[s[actual]]] ++ obtener_fila(s, s[actual])
  endif;

output [
  "Vector sig crudo: ", show(sig), "\n",
  "Primera persona:  ", show(personas[fix(sig[0])]), "\n",
  "Fila ordenada:    ", show(obtener_fila(fix(sig), 0)), "\n"
];
```

5.2.2. Restricciones Redundantes y Rompimiento de Simetrías

Restricciones Redundantes:

A fin de apoyar el proceso de propagación de restricciones se decidieron implementar las siguientes restricciones redundantes al interior de la formulación del modelo en Minizinc:

- **Distancia mínima de separate:** Para las restricciones impuestas a partir de las preferencia*separate* en lugar de simplemente establecer que la distancia entre personas debe ser $\neq 1$, es útil para el propagador establecer una desigualdad que divida el dominio de las variables, por lo que podemos establecer que la distancia debe ser ≥ 2, por medio entonces de la siguiente restricción redundante:

```
% Distancia mínima de separate
constraint forall(i in index_set_1of2(separate)) (
  abs(posiciones[separate[i,1]] - posiciones[separate[i,2]]) >= 2
);
```

- **Suma constante:** Dado que el arreglo solución `posiciones` debe contener obligatoriamente a todas las personas, la suma de sus elementos es también un valor constante igual a $\frac{N(N+1)}{2}$, por lo que podemos establecer esta restricción para apoyar al propagador a determinar los valores faltantes por ubicar en el arreglo:

```
% Preserva la suma constante de las posiciones
constraint sum(posiciones) = (N * (N + 1)) div 2;
```

Rompimiento de simetrías: Dado que no se establecen preferencias que definan un orden específico de las personas en la foto y tan sólo se limita la distancia relativa entre pares de personas, para una solución valida $(p_1, p_2, \cdots, p_n)$ su inversión $(p_n, p_{n-1}, \cdots, p_1)$ es igualmente válida, pues invertir el orden no altera la distancia entre personas. Esto efectivamente dobla el número de solución posibles sin obtener ninguna información nueva de la mitad de ellas.

Para romper esta simetrías se obliga que la primera persona siempre esté ubicada en la primera mitad de la fila, evitando así que el inverso de la fila sea también una solución válida, para ello obligamos al valor de $posicion[1]$ a ser menor a la mitad de la fila:

```
% La persona 1 siempre se ubica en la primera mitad de la fila.
constraint posiciones[1] < (N + 1) / 2;
```

Observe que se emplea el valor $(N + 1)/2$ para que la restricción se acomode tanto a listas de longitud par como impar. Con ello para el siguiente ejemplo con $n = 8$:

```
personas = ["jose", "julian", "diana", "juan",
    "david", "claudia", "edgar", "mario"];
next = [|5, 8|]; %David con Mario
separate = [|6, 7|]; %Claudia separada de Edgar
distance = [|6, 5, 2|]; %Claudia a 2 personas de David
```

El total de soluciones pasa de 4560 a exactamente 2280, reduciendo efectivamente el número de soluciones a la mitad, manteniendo la información total, reduciendo así enormemente los costos de búsqueda de soluciones.

5.3. Estrategias de Búsqueda y Distribución

Se determinaron explorar dos estrategias de búsqueda para comparar el desempeño alcanzado por el solver en cuestión de nodos explorados, fallados y tiempo de búsqueda:

- **Estrategia sencilla:** Se incluyeron las anotaciones `first_fail` para escoger primero las personas con menos posiciones posibles, y `indomain_min` para primero intentar ubicarlos en la posición más a la izquierda posible. Esto con el de reducir el overhead de proceso de selección de variables y valores para reducir costos para problemas de magnitud con N pequeños:

```
solve :: int_search(
posiciones,
first_fail,
indomain_min,
complete)
satisfy;
```

- **Estrategia compleja:** Se emplea la anotación `dom_w_deg` para que el solver identifique las personas con las restricciones más problemáticas y las ataque primero para fallar de forma temprana y reducir las ramas de búsqueda inútiles. Adicionalmente, la decisión de valores se hace con `indomain_split` para dividir el dominio de posiciones posibles a la mitad, lo que es útil para las preferencias de tipo *distance* pues se reduce rápidamente las cotas máximas y mínimas de las posiciones de las personas y por lo tanto la distancia entre las mismas:

```
solve :: int_search(
posiciones,
dom_w_deg,
indomain_split,
complete)
satisfy;
```

5.4. Pruebas y Análisis Experimental

Se probaron los desempeños de los pares de estrategias de búsqueda e instancias de implementación del modelo para evaluar su desempeño y hallar la mejor aproximación al problema. Cada instancia se probó para hallar todas las soluciones al ejemplo de tamaño $n = 8$:

```
personas = ["jose", "julian", "diana", "juan",
"david", "claudia", "edgar", "mario"];
next = [|5, 8|]; %David con Mario
separate = [|6, 7|]; %Claudia separada de Edgar
distance = [|6, 5, 2|]; %Claudia a 2 personas de David
```

Con ambas instancias se implementó rompimiento de simetrías de soluciones inversas y se ejecutó cada par *instancia-estrategia* un total de 10 veces para calcular el tiempo de ejecución promedio y su desviación estándar, condensando los resultado en la Tabla 11.

A partir de los resultados obtenidos, como ilustra la Tabla 11 la mejor aproximación a la solución del problema la logra la implementación de la instancia base con el arreglo de posiciones y la estrategia de búsqueda compleja con la selección de variables ponderada y división del dominio. Explorando tan sólo 4561 nodos y fallando en una única instancia. De igual forma, toma el menor tiempo de ejecución promedio de tan sólo 0.44 segundos. Esto se debe a que el problema favorece la búsqueda de las variables (personas) más problemáticas y la división de dominio permite acotar rápidamente las restricciones de tipo *Distance*.

Instancia	Estrategia	Tiempo (s)	Nodos explorados	Nodos fallidos	Profundidad Máxima
Base	Sencilla	0.4651427 ± 0.0483	7979	1710	12
Base	Compleja	0.4496473 ± 0.0358	4561	1	13
Alt	Sencilla	1.0091062 ± 0.051	6893	1167	12
Alt	Compleja	1.0209701 ± 0.0687	5719	580	16

Tabla 11: Comparación de rendimientos para distintas instancias y estrategias de búsqueda del problema de Reunión.

También es interesante el hecho de que en general todas las búsquedas con la implementación alternativa en forma de anillo toma el doble del tiempo de ejecución, dado que aunque su estructura simplifica la definición de las restricciones a partir de las preferencias *Next to* y *Separate*, se debe incluir restricciones adicionales y funciones auxiliares tanto para las restricciones *Distance* como para garantizar la integridad de camino de la solución. Estas validaciones adicionales terminan resultando en un mayor gasto de procesamiento en la búsqueda de soluciones por encima de la instancia base, que aunque requiere de operaciones auxiliares de restas y valor absoluto, el coste de estas operaciones es mucho menor a las requeridas en la implementación alternativa.

Esta diferencia entre ambas implementaciones se hace más evidente al comparar los árboles de búsqueda resultado de la solución de un ejemplo simplificado:

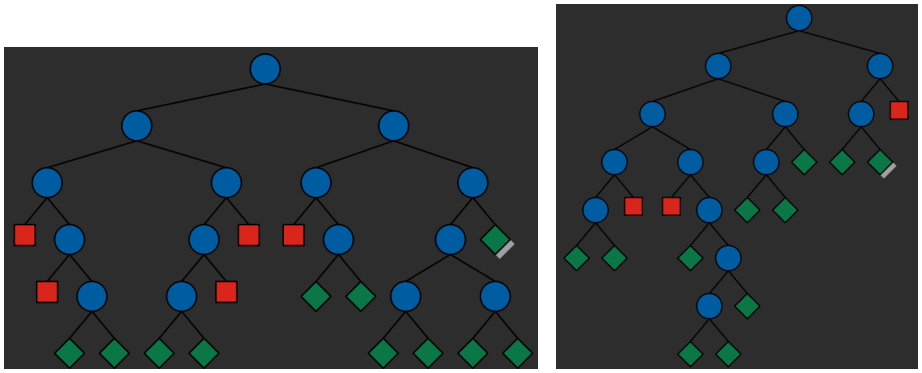
```
personas = ["jose", "david", "claudia", "edgar", "mario"];
next = [|2, 5|]; %David con Mario
separate = [|3, 4|]; %Claudia separada de Edgar
distance = [|3, 2, 2|]; %Claudia a 2 personas de David
```

Como evidencia la Figura 10 las instancias base y alternativas, aún al emplear la misma estrategia de búsqueda, resultan en árboles sumamente distintos, puesto que aprovechan aspectos distintos de las características de las restricciones del sistema. En este ejemplo en particular, la instancia base logra dividir los dominios más eficientemente, resultante en un árbol con menor profundidad, a diferencia de la instancia alternativa que requiere la expansión de más nodos y ramas de búsqueda más profundas para encontrar soluciones similares.

5.5. Conclusiones del Ejercicio

A partir de las implementaciones y distintas estrategias empleadas para el modelamiento y solución del problema de la reunión con programación por restricciones codificados en Minizinc, se concluyó que distintas aproximaciones al problema empleando distintas estructuras de datos resultan en la simplificación y mayor aprovechamiento de las características de los dominios y ciertas restricciones de los problemas, pero a la vez pueden añadir mayor complejidad a otros aspectos de los mismos. Por lo que es importante sopesar las ventajas y desventajas de todas las posibilidades de modelamiento del problema para emplear la de mejor desempeño, lo cual puede variar incluso según la naturaleza de las entradas a un mismo problema.

Adicionalmente, se evidenció el gran de poder de simplificación que otorga el rompimiento de simetrías para problemas de permutaciones, en dónde el espacio total de soluciones puede llegar a reducirse a la mitad o mucho más, sin perder información relevante sobre las soluciones



(a) Instancia Base (b) Instancia alternativa

Figura 10: Árboles de búsqueda para las instancias Base y Alternativa para la solución del ejemplo simplificado con estrategia de búsqueda compleja.

halladas. Esto reduce enormemente los tiempos de procesamiento casi de forma gratuita sin grandes desventajas.

6. Problema 6: Construcción de un Rectángulo

6.1. Descripción del Problema y Abstracción CSP

El problema de la construcción de un rectángulo consiste en ensamblar un conjunto dado de n cuadrados de lados enteros conocidos $s_1, s_2, \ldots, s_n$ para formar exactamente un rectángulo contenedor de anchura W y altura H. Para que una disposición sea válida, deben cumplirse tres condiciones geométricas:

1. **Integridad de uso:** Todos los n cuadrados deben ubicarse dentro del rectángulo contenedor con sus lados paralelos a los ejes coordenados.
2. **Contención:** Ningún cuadrado puede sobresalir de las fronteras $[0, W]\times[0, H]$ del rectángulo.
3. **No solapamiento:** Los interiores de dos cuadrados cualesquiera deben ser disjuntos (solo pueden compartir bordes o vértices).

Bajo el paradigma de Programación por Restricciones (CSP), si se sitúa el origen del sistema de coordenadas $(0, 0)$ en la esquina inferior izquierda del rectángulo contenedor, la posición de cada cuadrado $i \in \{1, \ldots, n\}$ queda completamente determinada por las coordenadas enteras (x_i, y_i) de su esquina inferior izquierda, ocupando la región espacial $[x_i, x_i + s_i] \times [y_i, y_i + s_i]$.

6.1.1. Parámetros y Datos de Entrada

El modelo paramétrico (`rectangulo.mzn`) recibe como entrada los siguientes datos desde los archivos `.dzn`:

- $n \in \mathbb{Z}^+$: Número total de cuadrados a ensamblar.
- $S = \{1, \ldots, n\}$: Conjunto de índices de los cuadrados.
- $s_i \in \mathbb{Z}^+$ (`lado[i]`): Longitud del lado del cuadrado $i \in S$. Por convención de modelado (verificada mediante `assert`), los cuadrados se suministran ordenados de mayor a menor tamaño: $s_1 \geq s_2 \geq \cdots \geq s_n$.
- $W \in \mathbb{Z}^+$: Anchura total del rectángulo contenedor.
- $H \in \mathbb{Z}^+$: Altura total del rectángulo contenedor.

6.1.2. Variables de Decisión y Dominios

Para cada cuadrado $i \in S = \{1, \ldots, n\}$ se definen dos variables de decisión enteras:

- x_i (`x[i]`): Coordenada horizontal (eje X) de la esquina inferior izquierda del cuadrado i.
- y_i (`y[i]`): Coordenada vertical (eje Y) de la esquina inferior izquierda del cuadrado i.

Dado que el cuadrado i se extiende s_i unidades hacia la derecha y s_i unidades hacia arriba, el dominio efectivo más estrecho para cada variable de decisión es:

$$D(x_i) = \{0, 1, \ldots, W - s_i\}, \qquad D(y_i) = \{0, 1, \ldots, H - s_i\}, \qquad \forall i \in \{1, \ldots, n\}$$

En MiniZinc, dado que la declaración de un arreglo requiere un rango de índice uniforme, los arreglos `x` e `y` se declaran sobre $0 \ldots W - \min_k(s_k)$ y $0 \ldots H - \min_k(s_k)$ respectivamente, e inmediatamente se acota el dominio exacto de cada componente mediante restricciones unarias.

6.1.3. Restricciones del Modelo

El modelo matemático base comprende las siguientes restricciones fundamentales:

1. **Restricciones unarias de contención exacta dentro del rectángulo $W \times H$:**

$$\forall i \in \{1, \ldots, n\}: \quad 0 \leq x_i \leq W - s_i \ \wedge \ 0 \leq y_i \leq H - s_i$$

2. **Restricción disyuntiva de no solapamiento bidimensional por pares:** Dos cuadrados distintos $i, j \in \{1, \ldots, n\}$ $(i < j)$ no se solapan si y solo si al menos una de las cuatro separaciones ortogonales (izquierda, derecha, abajo o arriba) se cumple:

$$\forall i, j \in \{1, \ldots, n\} \text{ con } i < j: \quad (x_i + s_i \leq x_j) \vee (x_j + s_j \leq x_i) \vee (y_i + s_i \leq y_j) \vee (y_j + s_j \leq y_i)$$

6.2. Detalles de Implementación (`rectangulo.mzn`)

Siguiendo el diseño modular de los problemas de planificación disyuntiva y mudanza vistos en el curso:

- **Predicado modular `no_solapar`:** La disyunción de cuatro cláusulas se encapsuló en un predicado reutilizable `no_solapar(x1, l1, y1, x2, l2, y2)`, invocándolo sobre el triángulo superior de pares $i < j$ para evitar duplicar restricciones entre el mismo par de cuadrados.
- **Ordenamiento decreciente de cuadrados y validación de área:** Mediante sentencias `assert` se verifica en tiempo de compilación que el arreglo `lado` cumpla $s_i \geq s_{i+1}$ y que la suma de áreas $\sum_{i=1}^{n} s_i^2$ coincida exactamente con el área $W \cdot H$ del contenedor.
- **Arreglo intercalado para búsqueda (`vars_xy`):** Se construyó por comprensión el arreglo unidimensional $[x_1, y_1, x_2, y_2, \ldots, x_n, y_n]$ de longitud $2n$, de modo que las heurísticas posicionales fijen de manera consecutiva ambas coordenadas (x_i, y_i) del cuadrado i antes de pasar al cuadrado $i + 1$.

6.2.1. Restricciones Redundantes y Rompimiento de Simetrías

Restricciones Redundantes. La restricción disyuntiva de no solapamiento $\vee$ sufre de propagación tardía (reificación débil): mientras un par de cuadrados conserve al menos dos direcciones posibles de separación, ninguna de las desigualdades individuales poda los dominios de x_i o y_i. Esto permite que el solver intente apilar varios cuadrados en una misma franja vertical u horizontal excediendo la altura H o la anchura W, descubriendo el fallo únicamente tras múltiples ramificaciones.

Para dotar al modelo de una visión global del espacio disponible, se observa que el empaquetamiento ortogonal 2D se proyecta sobre cada eje como un problema de planificación acumulativa de recursos (`cumulative`):

1. **Proyección acumulativa sobre el eje X:** Interpretando x_i como el instante de inicio de una tarea de duración s_i que consume una cantidad s_i del recurso vertical disponible, en toda abscisa $c \in [0, W)$ la suma de las alturas de los cuadrados que intersectan la recta vertical $X = c$ no puede exceder H (de hecho, como el recubrimiento es exacto, suma exactamente H):

$$\forall c \in \{0, \ldots, W-1\}: \quad \sum_{\substack{i=1 \\ x_i \leq c < x_i + s_i}}^{n} s_i \leq H \quad \iff \quad \text{cumulative}(x, s, s, H)$$

2. **Proyección acumulativa sobre el eje Y:** De forma dual, interpretando y_i como el inicio de una tarea de duración s_i y consumo horizontal s_i, en toda ordenada $r \in [0, H)$ la suma de anchuras de los cuadrados que intersectan la recta horizontal $Y = r$ no puede exceder W:

$$\forall r \in \{0, \ldots, H-1\} : \quad \sum_{\substack{i=1 \\ y_i \le r < y_i + s_i}}^{n} s_i \le W \quad \iff \quad \text{cumulative}(y, s, s, W)$$

Rompimiento de Simetrías y Recuperación de Soluciones. El problema posee dos fuentes independientes de simetría que multiplican exponencialmente el número de soluciones equivalentes:

1. **Simetría de permutación entre cuadrados de igual tamaño:**

   - *Simetría que rompe:* Si un subconjunto de k_g cuadrados tiene exactamente el mismo lado ($s_i = s_{i+1} = \ldots$), sus ubicaciones en cualquier solución válida pueden permutarse de $k_g!$ maneras distintas produciendo el mismo diseño geométrico. Por ejemplo, en la instancia del taller lado $= [3, 2, 2, 1, 1, 1]$, existen $2! \times 3! = 12$ permutaciones idénticas por cada configuración geométrica.
   - *Restricción impuesta:* Para todo par consecutivo de cuadrados de igual lado ($s_i = s_{i+1}$), se impone un orden lexicográfico estricto sobre sus vectores de posición $(x_i, y_i) <_{\text{lex}} (x_{i+1}, y_{i+1})$:

     $$\forall i \in \{1, \ldots, n-1\} \text{ tal que } s_i = s_{i+1} : \quad (x_i < x_{i+1}) \ \vee \ (x_i = x_{i+1} \wedge y_i < y_{i+1})$$

   - *Recuperación de las soluciones descartadas:* A partir de una solución canónica hallada, todas las soluciones equivalentes por reetiquetado se recuperan aplicando las $\prod_g k_g!$ permutaciones de índices dentro de cada clase de cuadrados de igual tamaño.

2. **Simetría geométrica dihedral del rectángulo contenedor:**

   - *Simetría que rompe:* Cualquier recubrimiento válido del rectángulo $W \times H$ permanece válido al aplicarle una reflexión horizontal respecto a $X = W/2$, una reflexión vertical respecto a $Y = H/2$, o una rotación de 180° (composición de ambas), generando 4 soluciones congruentes (y hasta 8 si el contenedor es cuadrado, $W = H$, al incluir la reflexión diagonal $X \leftrightarrow Y$).
   - *Restricción impuesta:* Se restringe el centro del cuadrado de mayor tamaño ($i = 1$) para que se ubique en el cuadrante inferior izquierdo del rectángulo contenedor:

     $$2x_1 \le W - s_1 \quad \wedge \quad 2y_1 \le H - s_1$$

     y adicionalmente, cuando el contenedor es cuadrado ($W = H$), se fuerza $x_1 \le y_1$ para romper la simetría de transposición diagonal.
   - *Recuperación de las soluciones descartadas:* Dada una solución canónica $\{(x_i, y_i)\}_{i=1}^{n}$, las configuraciones geométricas simétricas se reconstruyen mediante las isometrías afines:

     $$\begin{aligned} \text{Reflexión horizontal:} \quad & (x_i', y_i') = (W - s_i - x_i, \ y_i) \\ \text{Reflexión vertical:} \quad & (x_i', y_i') = (x_i, \ H - s_i - y_i) \\ \text{Rotación de } 180° : \quad & (x_i', y_i') = (W - s_i - x_i, \ H - s_i - y_i) \end{aligned}$$

     (y si $W = H$, componiendo las cuatro transformaciones anteriores con el intercambio de ejes (y_i, x_i)).

6.3. Estrategias de Búsqueda y Distribución

Para evaluar el impacto del orden de ramificación sobre el empaquetamiento 2D, se compararon seis estrategias de búsqueda:

1. **Default (`solve satisfy`):** Heurística nativa del solver Gecode.
2. **`input_order` + `indomain_min` sobre `vars_xy`:** Dado que `lado` está ordenado de mayor a menor ($s_1 \geq \cdots \geq s_n$) y `vars_xy` intercala $[x_1, y_1, \ldots, x_n, y_n]$, esta estrategia implementa la heurística geométrica clásica *Largest-Square-First, Bottom-Left*: posiciona primero los cuadrados más voluminosos lo más abajo y a la izquierda posible.
3. **`input_order` + `indomain_max` sobre `vars_xy`:** Ubica los cuadrados de mayor a menor intentando colocarlos en la esquina superior derecha disponible.
4. **`first_fail` + `indomain_min` sobre `vars_xy`:** Selecciona dinámicamente la coordenada x_i o y_i con menor dominio actual.
5. **`seq_search` por ejes (X luego Y) con `first_fail`:** Fija primero todas las abscisas $x_1, \ldots, x_n$ de todos los cuadrados y posteriormente resuelve sus ordenadas $y_1, \ldots, y_n$.
6. **`dom_w_deg` + `indomain_split` sobre `vars_xy`:** Combina el grado ponderado de fallos con bisección de dominios.

6.4. Pruebas y Análisis Experimental

Se diseñaron tres instancias de prueba en archivos `.dzn`:

- **`rectangulo1.dzn` (Instancia del enunciado del taller):** $W = 5, H = 4$ (Área = 20), $n = 6$ cuadrados con lado = $[3, 2, 2, 1, 1, 1]$. Tiene $2! \times 3! \times 4 = 48$ soluciones simétricas equivalentes a una única configuración geométrica salvo isometrías.
- **`rectangulo2.dzn` (Instancia mediana):** $W = 8, H = 5$ (Área = 40), $n = 9$ cuadrados con lado = $[3, 3, 2, 2, 2, 2, 2, 1, 1]$. Cada configuración geométrica genera $2! \times 5! \times 2! \times 4 =$ 1,920 soluciones simétricas.
- **`rectangulo3.dzn` (Instancia grande):** $W = 10, H = 7$ (Área = 70), $n = 12$ cuadrados con lado = $[4, 4, 3, 3, 2, 2, 2, 2, 1, 1, 1, 1]$.

Para la instancia del taller (`rectangulo1.dzn`, 5×4), el modelo obtiene la solución canónica única:

Cuadrado i	1 (3×3)	2 (2×2)	3 (2×2)	4 (1×1)	5 (1×1)	6 (1×1)
Posición (x_i, y_i)	$(0, 0)$	$(3, 0)$	$(3, 2)$	$(0, 3)$	$(1, 3)$	$(2, 3)$

$$\text{Mapa } (5 \times 4) = \begin{pmatrix} 4 & 5 & 6 & 3 & 3 \\ 1 & 1 & 1 & 3 & 3 \\ 1 & 1 & 1 & 2 & 2 \\ 1 & 1 & 1 & 2 & 2 \end{pmatrix}$$

En el Cuadro 12 se presenta el estudio de ablación al buscar todas las soluciones (`--all-solutions`, límite de 5,000 ms) bajo las cuatro combinaciones del modelo: (1) Base (solo `no_solapar`), (2) Base + Redundantes (`cumulative`), (3) Base + Rompimiento de Simetrías y (4) Modelo Completo. Los resultados demuestran el efecto multiplicativo de ambas técnicas: en `rectangulo1.dzn`, el rompimiento de simetrías reduce exactamente las 48 soluciones simétricas a la única solución canónica (1 solución), disminuyendo los nodos de 335 a 15. En `rectangulo2.dzn` y `rectangulo3.dzn`, donde los modelos sin rompimiento de simetrías agotan el tiempo límite de 5 s debido a la explosión factorial de permutaciones, el rompimiento de simetrías permite

enumerar la totalidad de las soluciones canónicas (8 y 122 respectivamente) en milisegundos, y la adición de `cumulative` reduce a menos de la mitad los nodos y fallos restantes (1,505 → 753 nodos en `rectangulo2.dzn` y 21,755 → 9,649 nodos en `rectangulo3.dzn`).

Instancia	Configuración del Modelo	Soluciones	Propagaciones	Nodos	Fallos	Tiempo (ms)
`rectangulo1.dzn`	1. Base (solo `no_solapar`)	48	6,146	335	120	1.37
($5 \times 4, n = 6$)	2. Base + Redundantes (`cumulative`)	48	5,920	255	80	1.48
	3. Base + Simetrías	1	413	17	8	0.14
	4. Completo (Redundantes + Simetrías)	**1**	**440**	**15**	**7**	**0.22**
`rectangulo2.dzn`	1. Base (solo `no_solapar`)	3,840*	43,645,037	657,009	324,660	>5,000
($8 \times 5, n = 9$)	2. Base + Redundantes (`cumulative`)	4,778*	564,913	15,461	2,948	>5,000
	3. Base + Simetrías	8	78,857	1,505	745	4.39
	4. Completo (Redundantes + Simetrías)	**8**	**50,329**	**753**	**369**	**4.16**
`rectangulo3.dzn`	1. Base (solo `no_solapar`)	1,986*	18,725,217	280,758	138,388	>5,000
($10 \times 7, n = 12$)	2. Base + Redundantes (`cumulative`)	2,375*	221,080	4,910	75	>5,000
	3. Base + Simetrías	122	1,254,766	21,755	10,756	114.01
	4. Completo (Redundantes + Simetrías)	**122**	**646,885**	**9,649**	**4,703**	**103.03**

Tabla 12: Estudio de ablación en búsqueda exhaustiva (`--all-solutions`, $*$ indica corte a los 5 s).

Asimismo, el Cuadro 13 compara las seis estrategias de búsqueda para encontrar la primera solución factible, contrastando el Modelo Base (Configuración 1) frente al Modelo Completo (Configuración 4).

		Modelo Base (Config. 1)			Modelo Completo (Config. 4)		
Instancia	Estrategia de Búsqueda	Tiempo (ms)	Nodos	Fallos	Tiempo (ms)	Nodos	Fallos
`rectangulo1.dzn`	Default (`solve satisfy`)	0.14	14	4	0.18	8	3
	`input_order + indomain_min`	0.13	13	3	**0.13**	**4**	**0**
	`input_order + indomain_max`	0.14	13	3	0.16	15	7
	`first_fail + indomain_min`	0.15	18	6	0.15	6	1
	`seq_search (x luego y, ff)`	0.19	35	14	0.19	**4**	**0**
	`dom_w_deg + indomain_split`	0.15	12	2	0.18	5	1
`rectangulo2.dzn`	Default (`solve satisfy`)	1.18	322	156	**0.28**	**18**	6
	`input_order + indomain_min`	0.29	69	29	0.28	20	5
	`input_order + indomain_max`	0.25	69	29	1.09	247	122
	`first_fail + indomain_min`	191.72	76,986	38,488	0.31	32	11
	`seq_search (x luego y, ff)`	521.30	238,330	119,160	0.30	29	10
	`dom_w_deg + indomain_split`	0.63	186	85	0.33	**18**	**4**
`rectangulo3.dzn`	Default (`solve satisfy`)	0.67	147	66	0.48	43	17
	`input_order + indomain_min`	0.63	135	60	**0.29**	**22**	**4**
	`input_order + indomain_max`	0.72	135	60	2.86	407	199
	`first_fail + indomain_min`	813.19	257,007	128,496	0.63	43	16
	`seq_search (x luego y, ff)`	2,795.07	807,419	403,704	0.58	51	19
	`dom_w_deg + indomain_split`	0.65	94	34	0.35	30	8

Tabla 13: Evaluación experimental de estrategias de búsqueda en colocación de cuadrados (primera solución).

6.5. Conclusiones del Ejercicio

- **Efecto crítico de la restricción global `cumulative`:** En el Modelo Base (que depende exclusivamente de las disyunciones binarias `no_solapar`), heurísticas como `first_fail` y `seq_search` sufren un estancamiento severo en `rectangulo3.dzn` (257,007 y 807,419 nodos respectivamente), ya que al fijar coordenadas en un solo eje sin verificar la capacidad ortogonal acumulada construyen configuraciones globalmente infactibles. Al activar

`cumulative` en X e Y, los nodos con `first_fail` caen de 257,007 a 39 (una reducción del 99,98 %).

- **Superioridad de la heurística *Largest-Square-First* (`input_order + indomain_min`):** Al ordenar los cuadrados de mayor a menor tamaño e intercalar $[x_i, y_i]$, la estrategia `input_order + indomain_min` junto con el modelo completo logra el mejor desempeño global (4 nodos y 0 fallos en `rectangulo1.dzn`, y solo 22 nodos y 4 fallos en `rectangulo3.dzn`). Por el contrario, `indomain_max` degrada el rendimiento en el modelo con rompimiento de simetrías porque intenta asignar valores altos de (x, y) que entran en conflicto directo con el orden lexicográfico ascendente $x_i \leq x_{i+1}$ y la restricción del cuadrante inferior izquierdo.

7. Conclusiones Generales

A lo largo del desarrollo de este taller se abordaron seis problemas combinatorios y lógicos de naturaleza diversa: rompecabezas numéricos matriciales (*Sudoku* y *Kakuro*), secuencias aritméticas autorreferenciales (*Secuencia Mágica*), deducción lógica multiatributo (*Acertijo Lógico*), ordenamiento lineal de permutaciones con preferencias espaciales (*Ubicación de Personas en una Reunión*) y empaquetamiento geométrico ortogonal bidimensional (*Construcción de un Rectángulo*).

A partir del modelamiento formal en el marco declarativo CSP($\mathcal{FD}$), su implementación rigurosa en MiniZinc y el análisis experimental sistemático de los árboles de búsqueda, propagaciones, nodos fallidos y tiempos de respuesta provistos por los solvers **Gecode** y **Chuffed**, se derivan las siguientes conclusiones fundamentales:

7.1. Expresividad del Modelado Declarativo y Formulación Incondicional

1. **Eliminación de reificación mediante la elección adecuada del dominio:** Uno de los aprendizajes metodológicos más valiosos del taller se manifestó en el *Acertijo Lógico* (Punto 2.4). Frente al diseño tradicional indexado por persona —que requería cuantificaciones disyuntivas e implicaciones condicionadas para vincular apellidos y gustos musicales—, la elección de las edades finitas $E = \{24, 25, 26\}$ como dominio común de enlace permitió expresar todas las pistas del problema como igualdades y desigualdades algebraicas directas e incondicionales. Esta reformulación pura eliminó las 18 variables booleanas auxiliares de semirreificación, reduciendo en un 87,1 % las propagaciones en el motor (de 70 a 9) y facilitando que el compilador resuelva el problema en el nodo raíz sin recurrir a bifurcaciones.

2. **Superioridad operativa de las restricciones globales:** En la totalidad de los problemas ensayados, el empleo de restricciones globales especializadas superó de forma contundente a las descomposiciones en redes de restricciones binarias elementales:

   - En *Kakuro* y *Sudoku*, sustituir la red de desigualdades por pares ($\neq$) por el propagador global `alldifferent` no solo compactó el tamaño de la representación en FlatZinc (reduciendo los propagadores activos entre un 28 % y un 49 %), sino que activó algoritmos de consistencia de hiper-arco basados en flujos y emparejamientos bipartitos capaces de detectar subconjuntos saturados de Hall antes de que las variables quedaran univaluadas.
   - En *Secuencia Mágica*, el uso de la restricción global `count` frente a la suma explícita de variables indicadoras booleanas evitó la introducción cuadrática de $\Theta(n^2)$ variables auxiliares, disminuyendo en un orden de magnitud la memoria del motor y el tiempo de resolución.

7.2. El Poder Operativo de las Restricciones Redundantes

Las restricciones redundantes demostraron ser el mecanismo más potente para compensar la miopía intrínseca de los propagadores locales de consistencia de arco:

1. **Proyección de invariantes globales sobre hiperplanos algebraicos:** En la *Secuencia Mágica* (Punto 2.3), mientras los propagadores individuales de conteo no pueden inferir que la suma total de elementos está acotada cuando las variables están libres, la adición conjunta de las invariantes diofánticas R_1 ($\sum x_i = n$) y R_2 ($\sum (i-1)x_i = 0$) colapsó el espacio de búsqueda. En el propio nodo raíz, la consistencia de cotas dedujo la cota estricta $x_i \leq \lfloor \frac{n-1}{i-1} \rfloor$, forzando dominio binario $\{0, 1\}$ para toda la segunda mitad de la secuencia. Este fenómeno redujo el crecimiento de nodos de cuadrático ($\Theta(n^2)$) a una ley

estrictamente lineal ($n + 1$ nodos para todo $n \geq 8$ par), alcanzando en $n = 100$ una aceleración temporal superior a tres órdenes de magnitud (2 802×, pasando de 4 007,51 ms a 1,43 ms) y recortando el 99,96 % de las propagaciones.

2. **Cotas analíticas extremas en tramos combinatorios:** En el *Kakuro* (Punto 2.2), las cotas combinatorias deducidas en función de la longitud del tramo L y la suma objetivo S acoplaron la restricción lineal de suma con la no repetición de dígitos, podando los extremos imposibles en el preprocesamiento y resolviendo tableros de hasta 8×8 en apenas 3 nodos y 1 fallo.

3. **Proyección acumulativa en empaquetamiento ortogonal:** En la *Construcción de un Rectángulo* (Punto 2.6), la restricción disyuntiva geométrica de no solapamiento sufre de propagación retardada. La formulación de restricciones redundantes acumulativas `cumulative` en los ejes X e Y proporcionó al solver una visión continua de la capacidad espacial, impidiendo que el motor explore ramas inviables donde la suma de perfiles de los bloques excede la altura H o la anchura W. Esta redundancia redujo en más del 99,98 % los nodos explorados bajo heurísticas dinámicas en instancias grandes.

7.3. Rompimiento de Simetrías: Supresión de la Explosión Factorial

El análisis de automorfismos permitió contrastar problemas donde el rompimiento de simetrías es indispensable frente a problemas donde no existe simetría o su imposición sería errónea:

1. **Eliminación masiva de estados isomórficos con recuperación formal:** En el problema del *Rectángulo* (Punto 2.6), convergieron dos fuentes ortogonales de simetría: las isometrías del contenedor (grupo diédrico D_2 o D_4) y las permutaciones internas de cuadrados gemelos ($\prod k_g!$). En la instancia mediana (8×5), no romper simetrías provocó el agotamiento del tiempo límite (> 5 s) tras expandir más de 650 000 nodos por cada diseño geométrico repetido 1 920 veces; en contraste, fijar el cuadrado mayor al cuadrante inferior izquierdo e imponer orden lexicográfico entre cuadrados iguales permitió encontrar las 8 soluciones canónicas en apenas 4,16 ms con 753 nodos. De forma análoga, en la *Reunión* (Punto 2.5), romper la simetría de inversión izquierda-derecha redujo el espacio de soluciones exactamente al 50 % (4 560 → 2 280 soluciones). En ambos casos, se comprobó que toda solución canónica preserva la completitud del modelo, pues las soluciones descartadas pueden reconstruirse formalmente mediante transformaciones algebraicas afines e inversiones posicionales.

2. **Certificación de ausencia de simetrías:** En *Kakuro*, *Secuencia Mágica* y el *Acertijo Lógico*, el análisis formal demostró que cada variable posee una identidad semántica única e intransferible ligada a pistas cruzadas o pesos posicionales. Se demostró que forzar órdenes canónicos en tales contextos eliminaría soluciones legítimas (volviendo artificialmente insatisfacibles instancias válidas), ratificando que el rompimiento de simetrías debe sustentarse siempre en un análisis matemático del grupo de invarianzas del problema.

7.4. Interacción No Lineal entre Modelado y Estrategias de Distribución

Uno de los resultados experimentales más significativos del taller es que **la mejor estrategia de búsqueda no puede seleccionarse de manera independiente al modelo**:

1. **Inversión heurística condicionada por la poda:** En la *Secuencia Mágica*, bajo el modelo base sin redundantes, `input_order` superó ampliamente a `first_fail` (55 frente a 117 nodos en $n = 10$) debido a que los dominios iniciales eran perfectamente homogéneos de tamaño n. Sin embargo, al incorporar la restricción redundante R_2, la poda selectiva en la

raíz creó una asimetría estructural en los dominios: bajo este nuevo panorama, `first_fail` pasó a ser estrictamente óptima en todas las escalas evaluadas (101 nodos en $n = 100$), mientras que `input_order` exploró el doble de nodos (199).

2. **Alineación entre orden físico y orden de instanciación:** En el empaquetamiento de rectángulos, la heurística *Largest-Square-First / Bottom-Left* (`input_order` + `indomain_min` sobre un vector intercalado $[x_i, y_i]$ con cuadrados ordenados decrecientemente) demostró ser insuperable (4 nodos en la instancia base), pues ataca primero los elementos de mayor volumen que condicionan el espacio global y ramifica de manera consistente con el anclaje del rompimiento de simetrías. Por el contrario, heurísticas como `indomain_max` generaron conflictos severos al intentar posicionar bloques en las esquinas opuestas a las impuestas por las restricciones canónicas.

3. **Invarianza bajo consistencia global completa:** Cuando la propagación es capaz de alcanzar consistencia global completa en la raíz (como ocurrió en el *Acertijo Lógico* con `alldifferent`), el espacio de búsqueda colapsa a profundidad 0, tornando idéntico el rendimiento de todas las heurísticas de selección de variables y valores (1 nodo y 0 fallos).

7.5. Síntesis Metodológica

El desarrollo integral de este taller consolida una metodología rigurosa para abordar problemas combinatorios complejos en Programación por Restricciones:

1. **Fase 1 (Abstracción y Dominios):** Formular el problema buscando la representación más directa posible, minimizando o eliminando reificaciones y acotando los dominios finitos *a priori* con base en las propiedades estructurales del problema.

2. **Fase 2 (Encapsulamiento Global):** Preferir sistemáticamente restricciones globales nativas (`alldifferent`, `cumulative`, `count`) frente a descomposiciones locales por pares, aprovechando los algoritmos polinomiales de filtrado especializado de los solvers modernos.

3. **Fase 3 (Refuerzo Algebraico):** Formular e incorporar invariantes redundantes independientes que conecten variables distantes y proyecten leyes globales de conservación, demostrando formalmente su redundancia lógica y midiendo su impacto de poda.

4. **Fase 4 (Simetrías y Canonicidad):** Identificar exhaustivamente los automorfismos de variables y valores, imponiendo restricciones canónicas que eliminen soluciones isomórficas y estableciendo el mecanismo explícito de reconstrucción de las soluciones omitidas.

5. **Fase 5 (Sintonización Heurística y Verificación):** Diseñar estrategias de distribución informadas por la geometría de los dominios podados y validar la corrección de los modelos mediante baterías sistemáticas de datos que incluyan casos borde, regímenes escalables e instancias infactibles.